

UMich EECS 498-00

Linear classifier

- 图像表示：张量格式（ $N \times C \times H \times W$ ：Batch $\times$ Channels $\times$ Height $\times$ Width）。

1. KNN

k-Nearest Neighbors, KNN 算法是一种基于实例的监督学习方法，用于分类和回归任务。其核心思想是：相似的数据点在特征空间中彼此靠近。

(1) 欧氏距离 (Euclidean Distance)

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

(2) 曼哈顿距离 (Manhattan Distance)

各维度绝对差之和：

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n |x_i - y_i|$$

(3) 闵可夫斯基距离 (Minkowski Distance)

广义距离（ $p = 1$ 时为曼哈顿， $p = 2$ 时为欧氏）：

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

(4) 余弦相似度 (Cosine Similarity)

衡量向量方向的相似性：

$$\text{sim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

2. 算法步骤

1. 计算距离：新样本与所有训练样本的距离。
 2. 选择 k 个最近邻：按距离升序排列，取前 k 个样本。
 3. 决策规则：
 - 分类：多数表决（多数类别为预测结果）。
 - 回归：取 k 个邻居目标值的均值。
-

线性分类器：

- $f(x; W, b) = Wx + b$ （得分函数）
 $(W \in \mathbb{R}^{D \times C})$: 权重矩阵, $(x \in \mathbb{R}^D)$: 展平后的图像向量

Weight Initialization

1. **NO** all zero initialization (->compute the same gradients and undergo the exact same parameter updates)
2. small random number, but not necessarily work strictly better, e.g.

```
1 w = 0.01* np.random.randn(D,H)
```

Regularization

- 限制模型复杂度，使其在训练集和测试集上都能表现良好。
 - 通过惩罚模型参数的大小，避免参数值过大
 - 防止模型过拟合
-

1. L2正则化 (Ridge回归)

- 定义：在损失函数中加入模型参数的L2范数（平方和）。
- 公式：

$$L_{\text{reg}} = L(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$$

其中：

- $L(\mathbf{w})$ 是原始损失函数（如均方误差、交叉熵等）。
 - $\|\mathbf{w}\|_2^2 = \sum_{i=1}^n w_i^2$ 是参数的L2范数。
 - λ 是正则化系数，控制正则化强度。
- 特点：
 - 使参数值趋近于0但不完全为0。
 - 适合处理共线性问题。

```
1 loss += reg * torch.sum(w*w)
2 dw += 2 * reg * w
```

2 L1正则化 (Lasso回归)

- 定义：在损失函数中加入模型参数的L1范数（绝对值和）。
- 公式：

$$L_{\text{reg}} = L(\mathbf{w}) + \lambda \|\mathbf{w}\|_1$$

其中：

- $\|\mathbf{w}\|_1 = \sum_{i=1}^n |w_i|$ 是参数的L1范数。
- 特点：
 - 使部分参数值完全为0，实现特征选择。
 - 适合高维数据，生成稀疏解。

3 Elastic Net

- 定义：结合L1和L2正则化。
- 公式：

$$L_{\text{reg}} = L(\mathbf{w}) + \lambda_1 \|\mathbf{w}\|_1 + \lambda_2 \|\mathbf{w}\|_2^2$$

- 特点:
 - 综合L1和L2的优点, 适合高维数据且具有共线性问题。
-

4 Dropout

一般在全连接层之后应用, 用于很多层全连接层

- 定义: 在训练过程中随机丢弃部分神经元, 防止模型过度依赖某些特定神经元。
- 优点:
 - a. 使拥有redundant representation, 避免co-adaption of features
 - b. 训练大量有相同权重的模型
- 实现:
 - 每次训练时, 以概率 p 随机将神经元的输出置为0。
 - 测试时, 使用所有神经元, 但将输出乘以 p 以保持期望值一致。

一、前向传播 (训练时):

(1)生成随机概率矩阵

$$\text{rand_matrix} \sim U(0, 1)$$

(例如, 使用 `torch.rand_like(x)` 或 `np.random.rand`)

(2) 二值化

将随机矩阵转换为二值掩码:

$$\text{mask} = \begin{cases} 1 & \text{if rand_matrix} \geq p \\ 0 & \text{otherwise} \end{cases}$$

(3)Inverted Dropout

$$y = \text{mask} \cdot x \cdot \frac{1}{1 - p}$$

- x : 输入向量
- mask : 二值掩码 (元素为 0 或 1, 按概率 p 生成)
- p : 丢弃率

train : out = x 关闭 Dropout 层, 不进行输出缩放

二、反向传播：

被丢弃的神经元的梯度为 0，保留的神经元的梯度按 $1/(1-p)$ 缩放。

- 特点：
- 减少神经元之间的共适应性，增强泛化能力。

阶段	操作	缩放
训练	随机丢弃神经元，保留的神经元输出放大 $1/(1 - p)$	是（自动或手动）
测试	所有神经元激活，输出保持原值	

网络层类型	推荐使用 DROPOUT?	说明
全连接层 (Dense)	✔ 强烈推荐	参数量大，易过拟合。通常在每个全连接层后添加，如 <code>FC → Dropout → ReLU</code> 。
卷积层 (CNN)	⚠ 选择性使用	参数共享降低过拟合风险。若需使用，建议在最后 1-2 个卷积层后添加。
循环层 (RNN/LSTM)	✔ 推荐	隐状态易过拟合，通常在循环层之间或输出前添加（如 <code>LSTM → Dropout</code> ）。
输入层	✔ 低概率使用	丢弃率 p 通常较低（如 0.1-0.2），防止输入信息丢失过多。
输出层	✘ 禁止	会导致预测不稳定，破坏输出概率分布。

数学视角

3.1 约束优化视角

正则化可以看作在原始优化问题中加入约束条件：

- L2正则化： $\|\mathbf{w}\|_2^2 \leq C$
- L1正则化： $\|\mathbf{w}\|_1 \leq C$

通过拉格朗日乘子法，将约束优化问题转化为无约束优化问题。

3.2 贝叶斯视角

- L2正则化等价于对参数施加高斯先验。
 - L1正则化等价于对参数施加拉普拉斯先验。
-

系数的选择

- λ 的作用：
 - λ 越大，正则化强度越大，模型越简单。
 - λ 过小，正则化效果不明显。
 - λ 过大，模型可能欠拟合。
 - 选择方法：
 - 交叉验证：通过网格搜索或随机搜索选择最优 λ 。
 - 经验值：根据任务和数据规模调整。
-

应用

5.1 线性回归

- L2正则化：Ridge回归。
- L1正则化：Lasso回归。

5.2 逻辑回归

- 加入L1或L2正则化，防止过拟合。

5.3 神经网络

- L2正则化：权重衰减（Weight Decay）。
- Dropout：随机丢弃神经元。
- 数据增强：通过变换数据增加多样性。

Loss function:

$$L = \frac{\sum_i L_i}{N} + R(W)$$

多类SVM损失 (Hinge Loss):

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

梯度计算

(1) 当 $j \neq y_i$ 时

- 若 $s_j - s_{y_i} + \Delta > 0$, 则 L_i 包含这一项, 梯度为:

$$\frac{\partial L_i}{\partial \mathbf{W}_j} = \mathbf{x}_i$$

- 若 $s_j - s_{y_i} + \Delta \leq 0$, 梯度为0

(2) 当 $j = y_i$ 时

- 正确类别的权重 $\mathbf{W}_{y_i}$ 出现在所有 $j \neq y_i$ 的项中, 梯度为:

$$\frac{\partial L_i}{\partial \mathbf{W}_{y_i}} = - \sum_{j \neq y_i} \mathbb{I}(s_j - s_{y_i} + \Delta > 0) \cdot \mathbf{x}_i$$

其中 $\mathbb{I}(\cdot)$ 是指示函数

综上, 若 $s_j - s_{y_i} + \Delta > 0$:

$d\mathbf{W}_j += \mathbf{x}_i$ (错误类别的梯度累加)

$d\mathbf{W}_{y_i} -= \mathbf{x}_i$ (正确类别的梯度累减)

```
1 def svm_loss(x, y):
2     """
3     Computes the loss and gradient using for multiclass SVM
4     classification.
5     Inputs:
6     - x: Input data, of shape (N, C) where x[i, j] is the score
7       for the jth
```

```

6         class for the ith input.
7         - y: Vector of labels, of shape (N,) where y[i] is the label
          for x[i] and
8         0 <= y[i] < C
9         Returns a tuple of:
10        - loss: Scalar giving the loss
11        - dx: Gradient of the loss with respect to x
12        """
13        N = x.shape[0]
14        correct_class_scores = x[torch.arange(N), y]
15        margins = (x - correct_class_scores[:, None] +
16        1.0).clamp(min=0.)
17        margins[torch.arange(N), y] = 0.
18        loss = margins.sum() / N
19        num_pos = (margins > 0).sum(dim=1)
20        dx = torch.zeros_like(x)
21        dx[margins > 0] = 1.
22        dx[torch.arange(N), y] -= num_pos.to(dx.dtype)
23        dx /= N
24        return loss, dx

```

Softmax:

cross_entropy

```

1 import torch.nn.functional as F
2
3 loss = F.cross_entropy(input, target, weight=None,
4 size_average=None, ignore_index=-100, reduce=None,
5 reduction='mean')

```

1. **input**: 模型的原始输出（未经过 Softmax），形状为 `(N, C)` 或 `(N, C, d1, d2, ...)`。
 - **N** 是批次大小（样本数），**C** 是类别数。
 - 对于图像分类，可能是 `(N, C, H, W)`（如语义分割）。
2. **target**: 真实标签，形状为 `(N)` 或 `(N, d1, d2, ...)`，数据类型为**整数**（每个元素是对应样本的类别索引，范围 `[0, C-1]`）。

3. **reduction**: 损失的聚合方式（默认 **'mean'**）:

- **'mean'**: 返回所有样本的平均损失。
- **'sum'**: 返回所有样本的损失总和。
- **'none'**: 返回每个样本的损失（形状与 **target** 一致）。

4. **ignore_index**: 指定一个标签值，计算损失时会忽略该标签对应的样本（常用于语义分割中的背景标签）

•
$$L_i = -\log \left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}} \right)$$

or equivalently

$$L_i = -f_{y_i} + \log \left(\sum_j e^{f_j} \right)$$

1. 每行scores减去max_scores，防止答案溢出

2. 计算概率向量probs:

$$p_i = \frac{e^{f_i}}{\sum_{j=1}^K e^{f_j}} \quad \text{for } i = 1, 2, \dots, K$$

3. 找出真实标签对应correct_probs

`probs[range(N), y]`

```
1 4.loss -= correct_loss.sum
2 $$Loss=-log(probs_{true\_class})$$
```

计算梯度

$$\frac{\partial L}{\partial z_i} = p_i - y_i$$

批量数据的情况

在实际应用中，我们通常处理的是批量数据。假设批量大小为 N ，输入矩阵为 $\mathbf{Z} \in \mathbb{R}^{N \times C}$ ，真实标签矩阵为 $\mathbf{Y} \in \mathbb{R}^{N \times C}$ ，Softmax 输出矩阵为 $\mathbf{P} \in \mathbb{R}^{N \times C}$ ，则损失函数关于输入矩阵 $\mathbf{Z}$ 的导数为：

$$\frac{\partial L}{\partial \mathbf{Z}} = \frac{\mathbf{P} - \mathbf{y}}{N}$$

其中，除法是逐元素相除， N 是批量大小，用于对梯度进行归一化。

综上所述，Softmax 损失函数关于输入的导数公式为 $\frac{\partial L}{\partial z_i} = p_i - y_i$ （单样本），在批量数据情况下为 $\frac{\partial L}{\partial \mathbf{Z}} = \frac{\mathbf{P} - \mathbf{Y}}{N}$ 。这个公式在神经网络的反向传播算法中非常重要，用于计算梯度并更新模型参数。

```
1 def softmax_loss(x, y):
2     # 减去该样本得分中的最大值
3     shifted_logits = x - x.max(dim=1, keepdim=True).values
4     # 计算指数和
5     z = shifted_logits.exp().sum(dim=1, keepdim=True)
6     # 计算对数概率，使用对数恒等式 log(exp(a) / exp(b)) = a - b
7     log_probs = shifted_logits - z.log()
8
9     # 通过指数运算将对数概率转换为概率
10    probs = log_probs.exp()
11
12    # 获取样本数量 N
13    N = x.shape[0]
14
15    # 计算损失值，使用交叉熵损失公式
16    loss = (-1.0 / N) * log_probs[torch.arange(N), y].sum()
17
18    # 复制概率矩阵，用于后续计算梯度
19    dx = probs.clone()
20    # 对于每个样本，将其真实标签对应的概率值减 1
21    dx[torch.arange(N), y] -= 1
22    dx /= N
```

L1 loss

用“绝对误差”衡量预测偏差

$$e = \hat{y} - y$$

L1 损失定义为误差的绝对值： $L_1 = |e| = |\hat{y} - y|$

对于包含 N 个样本的数据集，总 L1 损失通常取平均值（也可用总和，取决于是否除以样本数）： $L_1^{\text{total}} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$

$$\frac{\partial L_1}{\partial \hat{y}} = \begin{cases} 1 & \text{if } \hat{y} > y \quad (\text{即 } e > 0) \\ -1 & \text{if } \hat{y} < y \quad (\text{即 } e < 0) \\ \text{无定义} & \text{if } \hat{y} = y \quad (\text{即 } e = 0) \end{cases}$$

L2 loss /MSE

用“平方误差”衡量预测偏差

平均平方误差（MSE, Mean Squared Error）：

$$L_2^{\text{total}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

L2损失对预测值 ($\hat{y}$) 的导数为：

$$\frac{\partial L_2}{\partial \hat{y}} = 2e = 2(\hat{y} - y)$$

Focal loss

调整模型？, capacity, regularization

Fully connected layer

(or Dense Layer), 全连接层中, 每一个输入神经元都与每一个输出神经元相连, 这种连接方式使得网络能够对输入特征进行全面的组合与变换。全连接层主要负责对前面层提取的特征进行整合和转换, 将其映射到一个新的特征空间, 常用于分类或回归任务。将图像展平

在two layer neutral network 中即为 hidden layer

1.Forward propogation

数据从输入层经过全连接层得到输出的过程

- 输入: 一个向量 $\mathbf{X} \in \mathbb{R}^n$, 维度为 n 。
- 输出: 一个向量 $\mathbf{y} \in \mathbb{R}^m$, 维度为 m 。
- 权重矩阵: $\mathbf{W} \in \mathbb{R}^{m \times n}$, 其中 W_{ij} 表示第 i 个输出与第 j 个输入之间的权重。
- 偏置向量: $\mathbf{b} \in \mathbb{R}^m$, 其中 b_i 是第 i 个输出的偏置

全连接层的输出通过以下公式计算:

$$\mathbf{y} = \mathbf{WX} + \mathbf{b}$$

应用激活函数 f 得到最终输出矩阵 $\mathbf{a}$:

$$\mathbf{a} = f(\mathbf{y}) = f(\mathbf{WX} + \mathbf{b})$$

常用激活函数包括:

- ReLU: $f(x) = \max(0, x)$
- Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$

```
1 #forward
2 zeros = torch.zeros_like(x)
3 relu_out = torch.maximum(x, zeros)
4 #backward
5 dx = dout * (x>0)
```

Note

注意torch.max 和 torch.maximum区别

`torch.max` 用于返回输入张量中指定维度上的最大值以及对应的索引

`torch.maximum` 函数用于逐元素比较两个张量，并返回每个位置上的最大值

- Tanh: $f(x)=\tanh(x)$

2.Back propogation

trick: 注意每个tensor的 shape

用于计算梯度的过程，目的是为了更新网络的参数（weight, bias）以最小化损失函数。

1.计算输出层误差:

假设损失函数为 L ，输出层的激活值为 $\mathbf{a}$

梯度计算

- 损失对输出的梯度:

$$\frac{\partial L}{\partial \mathbf{y}} = \frac{\partial L}{\partial \mathbf{a}} \odot f'(\mathbf{y})$$

其中 $\odot$ 表示逐元素相乘， $f'(\mathbf{y})$ 是激活函数的导数。

- 损失对权重的梯度:

$$\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{y}} \mathbf{X}^T$$

- 损失对偏置的梯度:

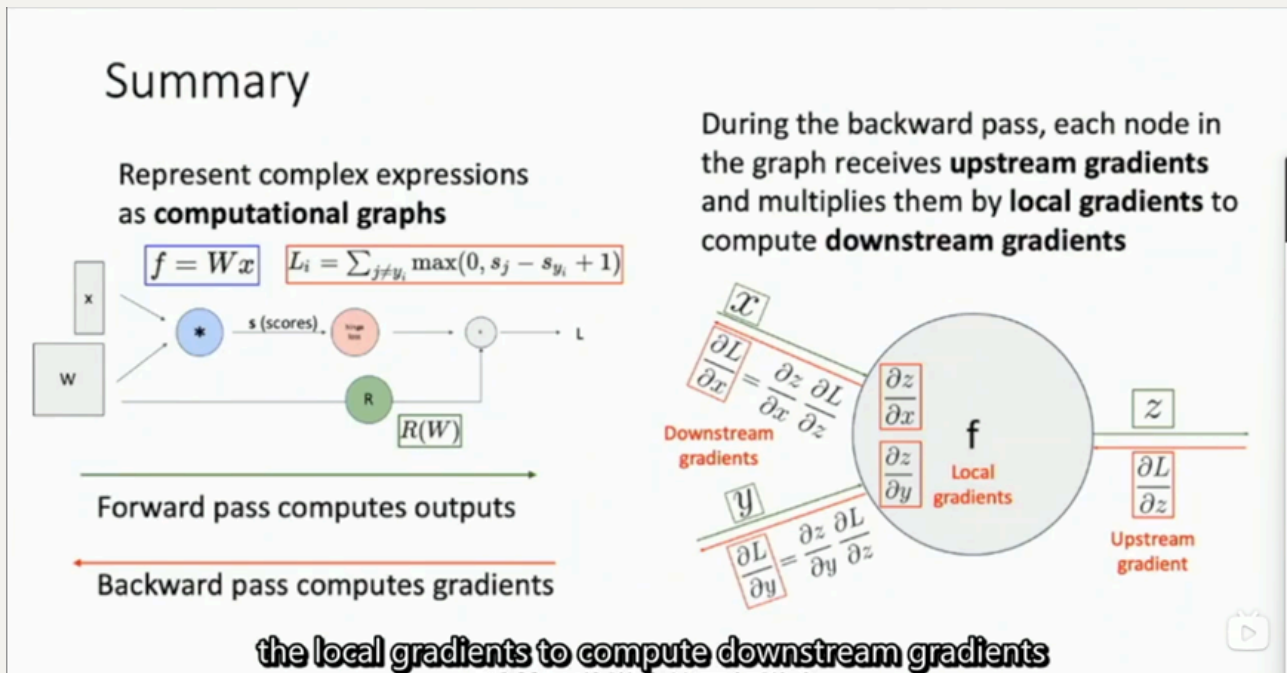
$$\frac{\partial L}{\partial \mathbf{b}} = \frac{\partial L}{\partial \mathbf{y}} = \sum_{i=1}^m L_i$$

其中 L_i 表示第 i 个样本对应的误差

```
1 #dout: Upstream derivative, of shape (N, M)
2 db = torch.sum(dout, dim=0)
```

- 损失对输入的梯度（用于前一层反向传播）:

$$\frac{\partial L}{\partial \mathbf{X}} = \mathbf{W}^T \frac{\partial L}{\partial \mathbf{y}}$$



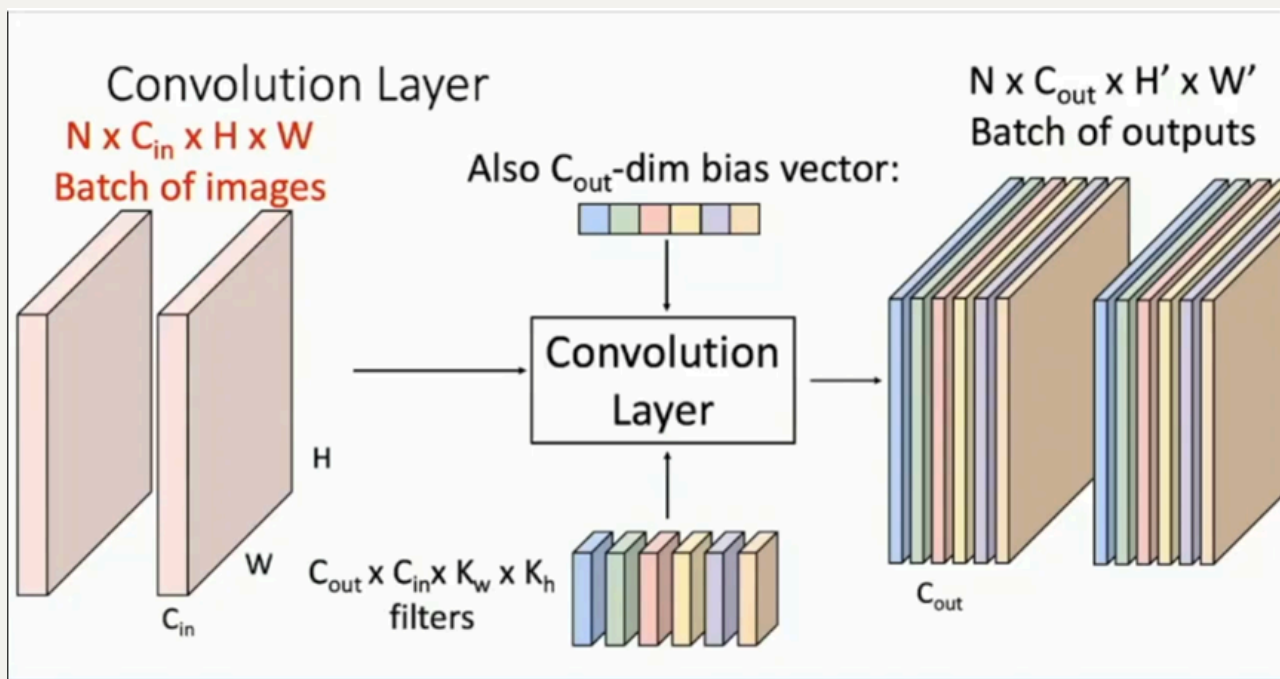
Convolution Neural Network

convolution layer:

提取局部特征，参数共享。

- 输入尺寸: C_{in}, H, W ,
- 滤波器filter / 卷积核 kernel: $(C_{out}, C_{in}, K_w, K_h)$
- 输出尺寸公式:

$$H_{out} = \left\lfloor \frac{H + 2P - K_h}{S} \right\rfloor + 1$$



Convolution Summary

Input: $C_{in} \times H \times W$

Hyperparameters:

- **Kernel size:** $K_H \times K_W$
- **Number filters:** C_{out}
- **Padding:** P
- **Stride:** S

Weight matrix: $C_{out} \times C_{in} \times K_H \times K_W$
giving C_{out} filters of size $C_{in} \times K_H \times K_W$

Bias vector: C_{out}

Output size: $C_{out} \times H' \times W'$ where:

- $H' = (H - K + 2P) / S + 1$
- $W' = (W - K + 2P) / S + 1$

Common settings:

$K_H = K_W$ (Small square filters)

$P = (K - 1) / 2$ ("Same" padding)

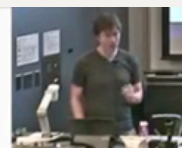
$C_{in}, C_{out} = 32, 64, 128, 256$ (powers of 2)

$K = 3, P = 1, S = 1$ (3x3 conv)

$K = 5, P = 2, S = 1$ (5x5 conv)

$K = 1, P = 0, S = 1$ (1x1 conv)

$K = 3, P = 1, S = 2$ (Downsample by 2)



parameters : $C_{out} * (C_{in} * H * W + 1(\text{bias}))$

floating-point operations(multiply + add) : $(C_{out} * H' * W') \times (C_{in} * K * K)$

```

1 #forward
2 #pad
3 x_pad = torch.nn.functional.pad(x, (pad, pad, pad, pad),
  mode='constant', value=0)
4 for... #循环遍历 计算receptive field
5     receptive_field = x_pad[n, :, h_start : h_start +
      HH, w_start : w_start + WW] #是逐元素相乘，不是矩阵乘法

```

```

6      #compute
7      out[n, f, h_out, w_out] = torch.sum(receptive_field * w[f])
      + b[f]
8
9      #backward
10     for f in range(F) :
11         db[f] = torch.sum(dout[:, f, :, :])
12     for....
13         dw[f] += receptive_field * dout[n, f, h_out, w_out]
14         dx_pad[n, :, h_start : h_start+HH, w_start : w_start + WW]
      += \
15         w[f] * dout[n, f, h_out, w_out]

```

Receptive fields

感受野是CNN中某一层神经元在输入图像上能感知到的区域大小。简单来说，它表示高层特征图中的单个神经元与输入图像的关联范围。随着网络层数加深，感受野逐渐扩大，使深层神经元能够捕捉更全局的特征。

每一层的感受野大小 R_l 和累积步长 S_l 由前一层决定：

$$R_l = R_{l-1} + (k_l - 1) \times S_{l-1}$$

$$S_l = S_{l-1} \times s_l$$

- R_{l-1} ：前一层感受野大小。
- k_l ：当前层卷积核或池化窗口大小。
- s_l ：当前层步长（Stride）。
- S_{l-1} ：前一层累积步长。

池化层Pooling layer:

主要用于降低特征图的空间维度（下采样），同时保留关键信息，增强模型的平移不变性和计算效率。 no learnable parameters

(1) 最大池化 (Max Pooling)

- 操作：在窗口内取最大值作为输出。

$$y_{i,j} = \max_{(p,q) \in \text{窗口}} x_{i+p,j+q}$$

- 特点：保留最显著特征，适合纹理、边缘等特征提取。

(2) 平均池化 (Average Pooling)

- 操作：在窗口内计算平均值作为输出。

- $$y_{i,j} = \frac{1}{k \times k} \sum_{(p,q) \in \text{窗口}} x_{i+p,j+q}$$

- 特点：平滑特征，适合背景信息较多的场景。

(3) 全局池化 (Global Pooling)

- 操作：对整个特征图进行池化，输出单个值（常用于分类层前）。
 - 全局最大池化：取整个特征图的最大值。
 - 全局平均池化：取整个特征图的平均值（替代全连接层，减少参数）。

(4) 自适应池化 (Adaptive Pooling)

- 操作：动态调整窗口大小，使输出为指定尺寸（如 `AdaptiveAvgPool2d(7)` 强制输出 7x7）。

Pooling Summary

Input: $C \times H \times W$

Hyperparameters:

- Kernel size: K
- Stride: S
- Pooling function (max, avg)

Common settings:

max, $K = 2, S = 2$

max, $K = 3, S = 2$ (AlexNet)

Output: $C \times H' \times W'$ where

- $H' = (H - K) / S + 1$
- $W' = (W - K) / S + 1$

Learnable parameters: None!

instead we apply some kind of fixed pooling function

相应地，我们应用某种固定函数的池化函数



floating-point ops for pooling layer : $(C_{out} * H' * W') \times (K * K)$

- 激活函数: ReLU (`nn.ReLU()`)、LeakyReLU (解决神经元“死亡”)。

1*1 convolution changes the number of channel dimensions while a **fully connected layer** (destroy the spatial structure) flats the whole tensor into one and then producing a vector output

Batch Normalization

随着网络层数的增加，每一层输入数据的分布会不断发生变化，这被称为“内部协变量偏移 (Internal Covariate Shift)”问题。内部协变量偏移会导致网络训练变慢，需要使用较小的学习率和精心设计的初始化方法，同时也容易出现梯度消失或梯度爆炸等问题。Batch Normalization的核心思想是在每一层的输入之前，对输入数据进行归一化处理，使其均值为0，方差为1，从而减少内部协变量偏移的影响。

Train time forward:

1. 计算均值，方差：

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

Per - channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

Per - channel std, shape is D

2. 归一化处理:

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

Normalized x, Shape is $N \times D$

其中, $\hat{x}_i$ 是归一化后的输入, ϵ 是一个很小的常数 (通常取 10^{-5} , 用于防止分母为零)。

3. 缩放和偏移 scale and shift :

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output, Shape is $N \times D$

$\gamma: (\mathbf{F}, _) \rightarrow \text{torch.ones}()$

$\beta: (\mathbf{F}, _) \rightarrow \text{torch.zeros}()$, where $\mathbf{F}$ is num_filters

其中, γ 和 β 是可学习的参数, 分别用于对归一化后的数据进行缩放和偏移。通过引入这两个参数, 网络可以根据自身的需求学习到合适的分布, 从而保留数据中的有用信息。

4. Update running mean and variance

```
1 running_mean = momentum * running_mean + (1-momentum) *  
  sample_mean  
2 running_var = momentum * running_var + (1-momentum) * sample_var
```

Test time forward :

```

1 # Use running mean and variance to normalize
2 x_hat = (x - running_mean) / torch.sqrt(running_var + eps)
3 out = gamma * x_hat + beta

```

Backward

设输入为 X ，输出为 $Y = \gamma \cdot \hat{X} + \beta$ ，其中：

$$\hat{X} = \frac{X - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad \mu = \frac{1}{N} \sum_i X_i, \quad \sigma^2 = \frac{1}{N} \sum_i (X_i - \mu)^2$$

(1) 参数梯度

$$\frac{\partial L}{\partial \gamma} = \sum_{i=1}^N \frac{\partial L}{\partial Y_i} \cdot \hat{X}_i, \quad \frac{\partial L}{\partial \beta} = \sum_{i=1}^N \frac{\partial L}{\partial Y_i}$$

```

1 dbeta = dout.sum(dim=0)
2 dgamma = torch.sum(dout * x_hat, dim=0)

```

(2) 输入梯度

链式法则展开：

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial \hat{X}} \cdot \frac{\partial \hat{X}}{\partial X} + \frac{\partial L}{\partial \mu} \cdot \frac{\partial \mu}{\partial X} + \frac{\partial L}{\partial \sigma^2} \cdot \frac{\partial \sigma^2}{\partial X}$$

- 计算 $\frac{\partial L}{\partial \hat{X}}$ ：

$$\frac{\partial L}{\partial \hat{X}} = \frac{\partial L}{\partial Y} \cdot \gamma$$

- 计算 $\frac{\partial L}{\partial \sigma^2}$ ：

$$\frac{\partial \hat{X}}{\partial \sigma^2} = -\frac{X - \mu}{2(\sigma^2 + \epsilon)^{3/2}}$$

$$\frac{\partial L}{\partial \sigma^2} = \sum_{i=1}^N \frac{\partial L}{\partial \hat{X}_i} \cdot \left(-\frac{X_i - \mu}{2(\sigma^2 + \epsilon)^{3/2}} \right)$$

- 计算 $\frac{\partial L}{\partial \mu}$ ：

$$\frac{\partial \hat{X}}{\partial \mu} = -\frac{1}{\sqrt{\sigma^2 + \epsilon}}, \quad \frac{\partial \sigma^2}{\partial \mu} = -\frac{2}{N} \sum_i (X_i - \mu)$$

$$\frac{\partial L}{\partial \mu} = \sum_{i=1}^N \frac{\partial L}{\partial \hat{X}_i} \cdot \left(-\frac{1}{\sqrt{\sigma^2 + \epsilon}} \right) + \frac{\partial L}{\partial \sigma^2} \cdot \left(-\frac{2}{N} \sum_i (X_i - \mu) \right)$$

- 最终输入梯度：

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial \hat{X}} \cdot \frac{1}{\sqrt{\sigma^2 + \epsilon}} + \frac{\partial L}{\partial \sigma^2} \cdot \frac{2(X - \mu)}{N} + \frac{\partial L}{\partial \mu} \cdot \frac{1}{N}$$

Spatial Batch Norm

在卷积层上应用，输入数据x 的形状为(N, C, H, W)

- (N)：样本数
- (C)：通道数
- (H)：特征图的高度
- (W)：特征图的宽度
- 对每个通道 (C) 的所有像素点（跨 (N, H, W)）进行归一化。
- 计算均值和方差时，视每个通道的所有像素点为一个整体。

实现步骤

1. 将输入数据 [x] 的形状从 (N, C, H, W) 转换为 (N * H * W, C)，以便使用之前实现的 [BatchNorm]。
2. 调用 [BatchNorm.forward] 对数据进行归一化。
3. 将输出数据的形状从 (N * H * W, C) 转换回 (N, C, H, W)。

Kaiming Normalization

Kaiming Normalization 核心思想是通过调整权重初始化的方差，保持前向传播和反向传播过程中各层信号的方差稳定，从而避免梯度消失或爆炸问题

核心原理

1. **方差一致性**: 确保网络各层输出的方差在正向传播时保持一致, 梯度在反向传播时也保持稳定。
2. **激活函数影响**: 针对ReLU的特性 (将负值置零), 调整初始化方差以补偿激活函数导致的方差变化。

1. 前向传播:

- 设输入为 x , 权重为 W , 线性变换后输出为 $y = Wx$ 。
- x 的方差: σ_x^2 , W 的方差: σ_W^2 。
- 线性变换后的方差为 $\text{Var}(y) = n_{\text{in}} \cdot \sigma_W^2 \cdot \sigma_x^2$, 其中 n_{in} 是输入神经元数。
- 经过ReLU后, 输出的方差变为 $\frac{1}{2} \text{Var}(y)$ (因一半神经元被抑制)。
- 为保持方差一致:

$$\frac{1}{2} \cdot n_{\text{in}} \cdot \sigma_W^2 \cdot \sigma_x^2 = \sigma_x^2$$

解得: $\sigma_W^2 = \frac{2}{n_{\text{in}}}$, 即标准差为 $\sqrt{\frac{2}{n_{\text{in}}}}$ 。

2. 反向传播:

- 类似地, 梯度方差需保持稳定。若使用输出神经元数 n_{out} , 则 $\sigma_W^2 = \frac{2}{n_{\text{out}}}$ 。

初始化公式

- **正态分布**: 权重初始化为均值为0, 标准差为 $\sqrt{\frac{2}{n}}$, 其中 n 为 `fan_in` (前向传播) 或 `fan_out` (反向传播)。
- **均匀分布**: 范围 $[-c, c]$, 其中 $c = \sqrt{\frac{6}{n}}$ 。
- **Leaky ReLU调整**: 若激活函数为Leaky ReLU (负斜率 a), 则方差调整因子为 $\frac{2}{(1+a^2)}$, 标准差为 $\sqrt{\frac{2}{(1+a^2) \cdot n}}$ 。

具体实现

1. 全连接层:

- `fan_in` = 输入神经元数, `fan_out` = 输出神经元数。
- 权重初始化: $\mathcal{N}(0, \sqrt{\frac{2}{\text{fan_in}}})$ 。
- size:(Din, Dout)

2. 卷积层:

- `fan_in` = 输入通道数 × 卷积核宽 × 卷积核高 (如 $C_{\text{in}} \times 3 \times 3$)。
- 权重初始化: $\mathcal{N}(0, \sqrt{\frac{2}{C_{\text{in}} \times k \times k}})$, 其中 k 为卷积核大小。
- size : (Dout, Di)

3. 偏置项: 通常初始化为0。

模式选择 (Mode)

- `fan_in` (默认): 保证前向传播的方差稳定, 适用于大多数情况。
- `fan_out`: 保证反向传播的梯度稳定, 可根据需求选择。

```
1 import torch.nn as nn
2
3 # 全连接层
4 linear = nn.Linear(in_features=100, out_features=200)
5 nn.init.kaiming_normal_(linear.weight, mode='fan_in',
6 nonlinearity='relu')
7
8 # 卷积层
9 conv = nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3)
10 nn.init.kaiming_normal_(conv.weight, mode='fan_in',
11 nonlinearity='leaky_relu', a=0.1)
```

-
- **优势:** 有效缓解梯度消失/爆炸, 支持深层网络 (如ResNet、VGG) 训练。
 - **适用场景:** 使用ReLU、Leaky ReLU等激活函数的网络, 尤其是深度模型。

经典架构对比

模型	核心创新	PYTORCH实现模块
ResNet	残差连接（Skip Connection）	<code>torchvision.models.resnet50()</code>
EfficientNet	复合缩放 (Depth/Width/Resolution)	<code>efficientnet_pytorch</code> 库
Vision Transformer (ViT)	将图像分块输入Transformer	<code>timm</code> 库中的 <code>vit_base_patch16</code>

代码示例（自定义ResNet Block）

```
1 class ResidualBlock(nn.Module):
2     def __init__(self, in_channels, out_channels, stride=1):
3         super().__init__()
4         self.conv1 = nn.Conv2d(in_channels, out_channels,
5                                 kernel_size=3, stride=stride, padding=1)
6         self.bn1 = nn.BatchNorm2d(out_channels)
7         self.conv2 = nn.Conv2d(out_channels, out_channels,
8                                 kernel_size=3, padding=1)
9         self.bn2 = nn.BatchNorm2d(out_channels)
10        self.shortcut = nn.Sequential() # 跳跃连接
11        if stride != 1 or in_channels != out_channels:
12            self.shortcut = nn.Sequential(
13                nn.Conv2d(in_channels, out_channels,
14                           kernel_size=1, stride=stride),
15                nn.BatchNorm2d(out_channels)
16            )
17
18    def forward(self, x):
19        out = F.relu(self.bn1(self.conv1(x)))
20        out = self.bn2(self.conv2(out))
21        out += self.shortcut(x)
22        return F.relu(out)
```

Optimization

优化算法

- **SGD with Momentum:**

$$v_{t+1} = \beta v_t + (1 - \beta) \nabla_{\theta} L(\theta_t)$$
$$\theta_{t+1} = \theta_t - \alpha v_{t+1}$$

v : 速度变量, velocity: A numpy array of the same shape as w and dw used to store a moving average of the gradients.

β : 动量系数 (通常设为0.9), 控制历史梯度的衰减速度

α : learning rate

```
1 v = 0
2 for t in range(num_steps) :
3     dw = compute_gradient(w)
4     # Momentum update
5     v = mu * v - learning_rate * dx # integrate velocity
6     x += v # integrate position
```

- **RMSprop (Root Mean Square Propagation)**

1. 计算梯度平方的指数移动平均:

$$E[g^2]_t = \rho \cdot E[g^2]_{t-1} + (1 - \rho) \cdot g_t^2$$

- ρ : 衰减率 (通常设为 0.9)
- g_t : 当前时刻的梯度

2. 更新参数:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} \cdot g_t$$

- η : 全局学习率

- ϵ : 小常数 (通常设为 10^{-8}), 防止分母为零

```
1 cache = decay_rate * cache + (1 - decay_rate) * dx**2
2 x += - learning_rate * dx / (np.sqrt(cache) + epsilon)
```

• Adam:

结合momentum与自适应学习率 (

默认参数: $\epsilon = 1e-8, \beta_1 = 0.9, \beta_2 = 0.999$).

a. 计算梯度的一阶矩 (动量) 和二阶矩 (自适应学习率):

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t \quad (\text{一阶矩, 类似动量})$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2 \quad (\text{二阶矩, 类似RMSprop})$$

```
1 2. **偏差校正 (Bias Correction) **:
2    由于初始时刻 $ m_0 = 0, v_0 = 0 $, 需校正早期估计的偏差:
```

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

```
1 3. **更新参数**:
```

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- η : 全局学习率
- ϵ : 小常数 (通常设为 10^{-8}), 防止分母为零

```
1 m = beta1*m + (1-beta1)*dx
2 v = beta2*v + (1-beta2)*(dx**2)
3 x += - learning_rate * m / (np.sqrt(v) + eps)
```

bias correction mechanism

```

1 # t is your iteration counter going from 1 to infinity
2 m = beta1*m + (1-beta1)*dx
3 mt = m / (1-beta1**t)
4 v = beta2*v + (1-beta2)*(dx**2)
5 vt = v / (1-beta2**t)
6 x += - learning_rate * mt / (np.sqrt(vt) + eps)

```

```

1 torch.optim.Adam(
2     params,                # 待优化的参数（通常是模型的parameters()）
3     lr=0.001,              # 初始学习率（默认0.001，可根据任务调整）
4     betas=(0.9, 0.999),    # 一阶矩和二阶矩的指数衰减率（默认(0.9,
                             # 0.999)）
5     eps=1e-08,             # 数值稳定性参数，避免分母为0（默认1e-8）
6     weight_decay=0,         # 权重衰减（L2正则化系数，默认0，即不使用）
7     amsgrad=False          # 是否使用AMSGrad变体（默认False，通常无需开
                             # 启）
8 )

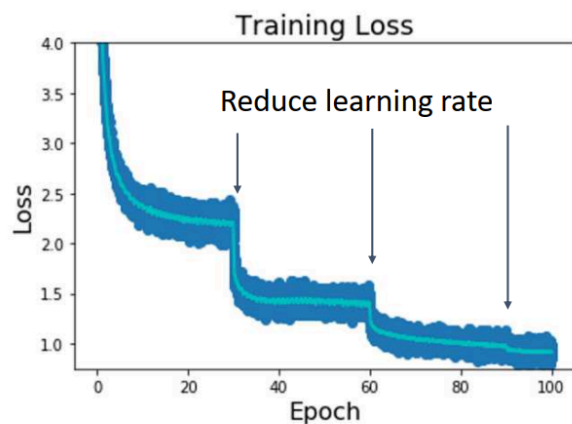
```

优化器	核心机制	优点	缺点
AdaGrad	累积历史梯度平方	适合稀疏数据	学习率过早衰减至零
RMSprop	指数加权平均历史梯度平方	自适应学习率，缓解AdaGrad的问题	超参数（如 ρ ）敏感
Adam	结合动量（一阶矩）和RMSprop（二阶矩）	适应性强，收敛快	需要调参较多

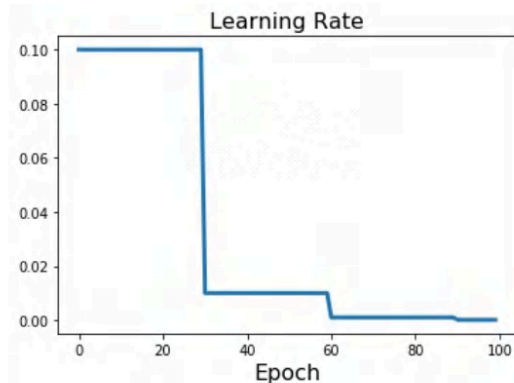
Learning rate schedule

1.Step

Learning Rate Decay: Step



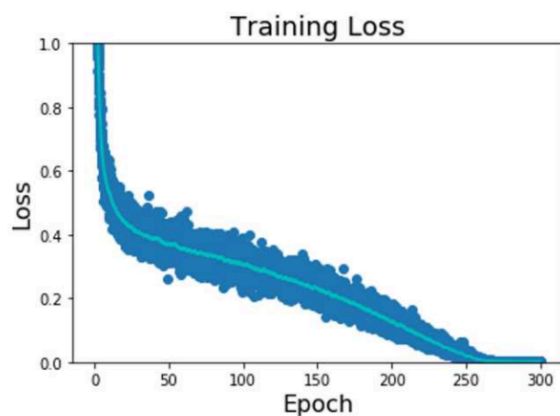
Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.



2.余弦退火 (Cosine Annealing):

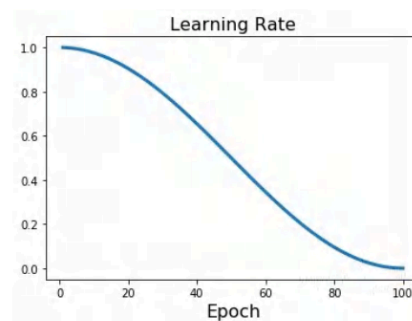
没有引入新的超参数，调整初始学习率和训练Epoch：T

Learning Rate Decay: Cosine



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine:
$$\alpha_t = \frac{1}{2} \alpha_0 \left(1 + \cos \left(\frac{t\pi}{T} \right) \right)$$



Loshchilov and Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts", ICLR 2017
Radford et al, "Improving Language Understanding by Generative Pre-Training", 2018
Feichtenhofer et al, "SlowFast Networks for Video Recognition", ICCV 2019
Radosavovic et al, "On Network Design Spaces for Visual Recognition", ICCV 2019
Child et al, "Generating Long Sequences with Sparse Transformers", arXiv 2019

3.Linear Decay

$$\alpha_t = \alpha_0(1 - t/T)$$

4.Inverse Sqrt

$$\alpha_t = \alpha_0 / \sqrt{t}$$

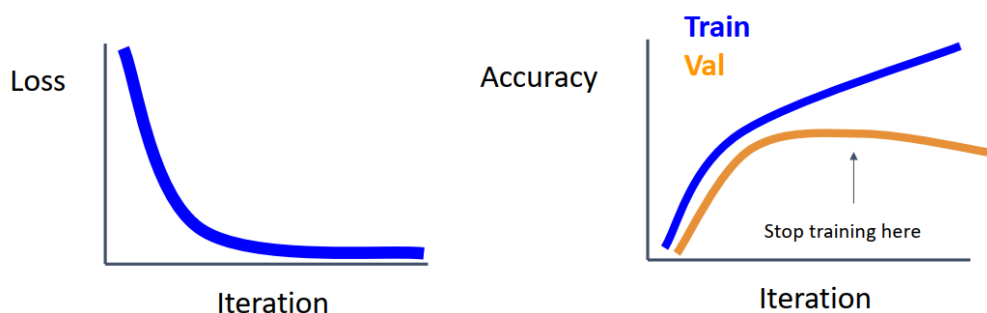
5.Constant $\alpha_t = \alpha_0$

How long to train?

Early Stopping

- 定义：在训练过程中监控验证集误差，当误差不再下降时提前停止训练。
- 特点：
 - 防止模型在训练集上过拟合。
 - 简单有效，无需修改损失函数。

How long to train? Early Stopping



Stop training the model when accuracy on the validation set decreases
Or train for a long time, but always keep track of the model snapshot that worked best on val. **Always a good idea to do this!**

Recurrent Neural Network

RNN处理序列的几种模式

- 一对多 (One to Many): 输入一个单一数据，输出一个序列。

- **例子：图片描述生成 (Image Captioning)**。输入一张图片，模型生成一句描述这张图片的话（一个词语序列）。
- **多对一 (Many to One)**：输入一个序列，输出一个单一数据。
 - **例子：视频分类**。输入一段视频（一个图像帧序列），模型判断视频的类别（如“体育”、“电影”）。
- **多对多 (Many to Many)**：输入和输出都是序列。
 - **例子1（长度可能不同）：机器翻译**。输入一句英文（词语序列），模型输出一句中文（另一个词语序列）。
 - **例子2（长度相同）：逐帧视频分类**。对视频的每一帧都进行分类。

2. RNN的核心思想：引入“记忆”

RNN之所以能够处理序列，关键在于它拥有一个**内部状态 (Internal State) i.e.隐藏状态 (Hidden State, h)**，用于存储到当前时间步为止的信息摘要。

核心工作流程： 在每个时间步（time step） t ，RNN会接收两个输入：

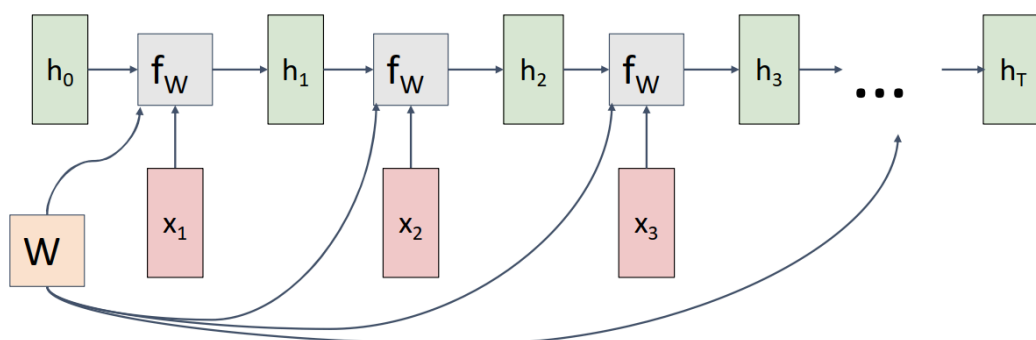
1. 当前时间步的输入数据 x_t 。
2. 上一个时间步的隐藏状态 h_{t-1} （即过去的“记忆”）。

$$h_t = f_W(h_{t-1}, x_t)$$

where f_W : some function with parameters W ，在所有的时间步中，这个函数 f_W 以及它的参数 W 都是共享的。这意味着RNN用同一套规则来处理序列中的每一个元素，从而大大减少了模型的参数量。

RNN Computational Graph

Re-use the same weight matrix at every time-step



Vanilla RNN

(Elman RNN) 计算公式如下：

1. 更新隐藏状态：

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- h_{t-1} ：上一个时间步的隐藏状态。
- x_t ：当前时间步的输入。
- W_{hh}, W_{xh}, b_h ：模型的权重和偏置，是需要学习的参数。
- **tanh**：激活函数（双曲正切），将输出值压缩到-1到1之间。

2. 计算输出：

$$y_t = W_{hy}h_t + b_y$$

- h_t ：当前时间步的隐藏状态。
- y_t ：当前时间步的输出。
- W_{hy}, b_y ：输出层的权重和偏置。

Images → ImageEncoder → Features



FeatureProjection → h0/A

线性变换层，用于将图像特征投影到适合RNN使用的形式



Captions → WordEmbedding → WordVectors



RNN/LSTM/Attention + h0/A → HiddenStates



OutputProjection → Scores

将RNN的隐藏状态转换为词汇表上的概率分布



TemporalSoftmax → Loss

时间反向传播 (BPTT)

RNN的训练过程是通过**时间反向传播 (Backpropagation Through Time, BPTT)** 来完成的。

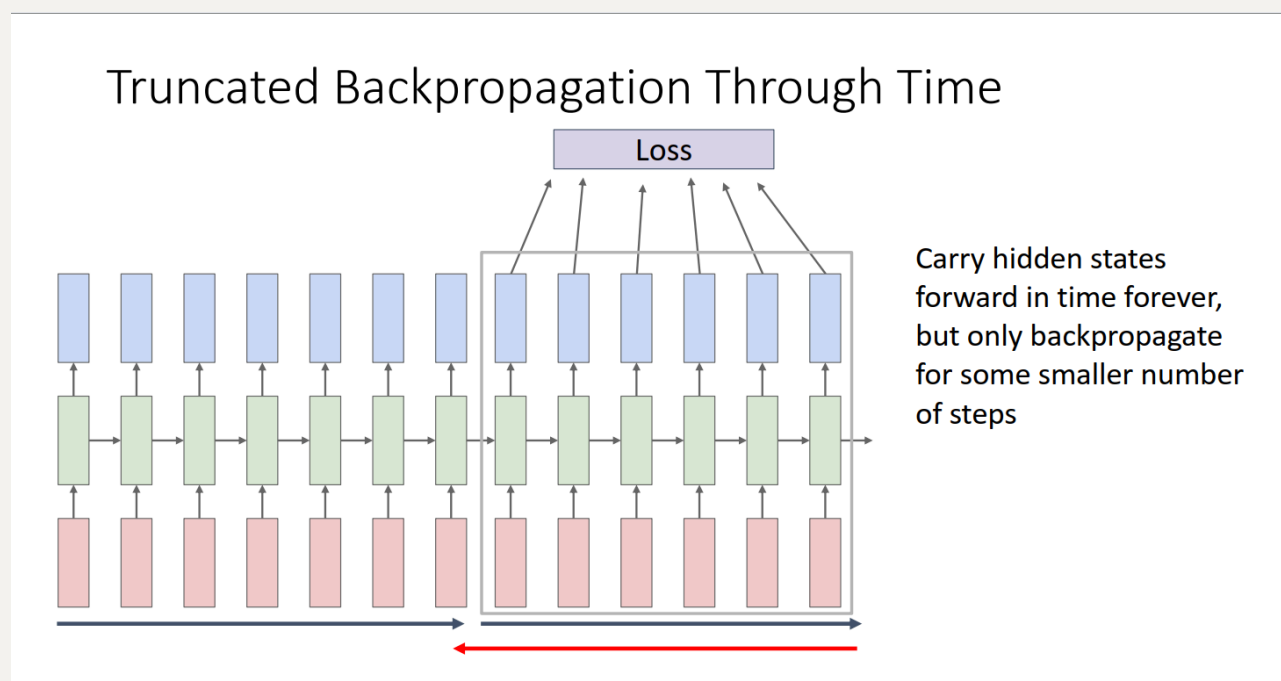
1. **前向传播**: 将整个输入序列 (例如, 一句话) 输入到RNN中, 从第一个时间步到最后一个时间步, 计算出每个时间步的输出和最终的损失 (Loss)。
2. **反向传播**: 将损失从最后一个时间步开始, 沿着展开的计算图一路反向传播回去, 计算出所有共享权重 (W_{hh}, W_{xh}, W_{hy} 等) 的梯度。

$$\partial L / \partial h_t = \partial L_{\text{直接}} / \partial h_t + \partial L_{\text{传播}} / \partial h_t$$

3. **参数更新**: 使用梯度下降法更新权重。

挑战: 对于非常长的序列, 将整个序列展开进行BPTT会占用巨大的内存。

解决方案: **截断时间反向传播 (Truncated BPTT)**。我们将序列分成若干个小块 (chunks), 每次只在一个小块内进行前向和反向传播。虽然反向传播被截断了, 但隐藏状态 (“记忆”) 会一直传递下去, 使得模型仍然能学习到长距离的依赖关系。



4. Problems: 梯度消失与梯度爆炸

RNN在理论上可以捕捉长距离依赖，但实践中却很困难。问题出在反向传播上。从公式 $h_t = \tanh(W_{hh}h_t - 1 + \dots)$ 可以看出，在反向传播时，梯度会反复乘以权重矩阵 W_{hh} 。

- **梯度爆炸 (Exploding Gradients)**: 如果 W_{hh} 的某些值比较大，经过多次连乘后，梯度会变得非常大，导致模型训练不稳定。
 - **解决方案: 梯度裁剪 (Gradient Clipping)**。设定一个阈值，如果梯度的范数超过这个阈值，就按比例缩小它。这是一个简单有效的技巧。
- **梯度消失 (Vanishing Gradients)**: 如果 W_{hh} 的某些值比较小（例如小于1），经过多次连乘后，梯度会趋近于零。这使得模型无法学习到序列早期部分的信息，即失去了“长期记忆”的能力。这个问题更严重，也更难解决。

长短期记忆网络 (LSTM)

为了解决梯度消失问题，长短期记忆网络 (**Long Short-Term Memory, LSTM**) 被提了出来。LSTM是RNN的一种特殊变体，结构更复杂，但效果好得多，至今仍是处理序列问题的首选模型之一。

LSTM的核心思想：门控机制

LSTM引入了一个新的核心组件：**细胞状态 (Cell State, c_t)**。可以把它想象成一条“信息传送带”，信息可以在上面直流，只有一些微小的线性交互。这使得梯度能够非常顺畅地在长序列中传递。

LSTM通过三个精密的“**门 (Gate)**”来控制细胞状态中信息的增加与移除。门是一种让信息选择性通过的结构，由一个Sigmoid激活函数和逐元素相乘操作组成。Sigmoid的输出在0到1之间，0表示“完全不允许通过”，1表示“完全允许通过”。

这三个门分别是：

1. **遗忘门 (Forget Gate, f_t)**: 决定从上一个细胞状态 c_{t-1} 中丢弃哪些信息。
2. **输入门 (Input Gate, i_t)**: 决定让哪些新的信息存入细胞状态 c_t 。
3. **输出门 (Output Gate, o_t)**: 决定从细胞状态 c_t 中输出哪些信息作为当前时间步的隐藏状态 h_t 。

**** LSTM的计算步骤****

下图展示了LSTM单元的内部结构：

其数学公式可以概括为：

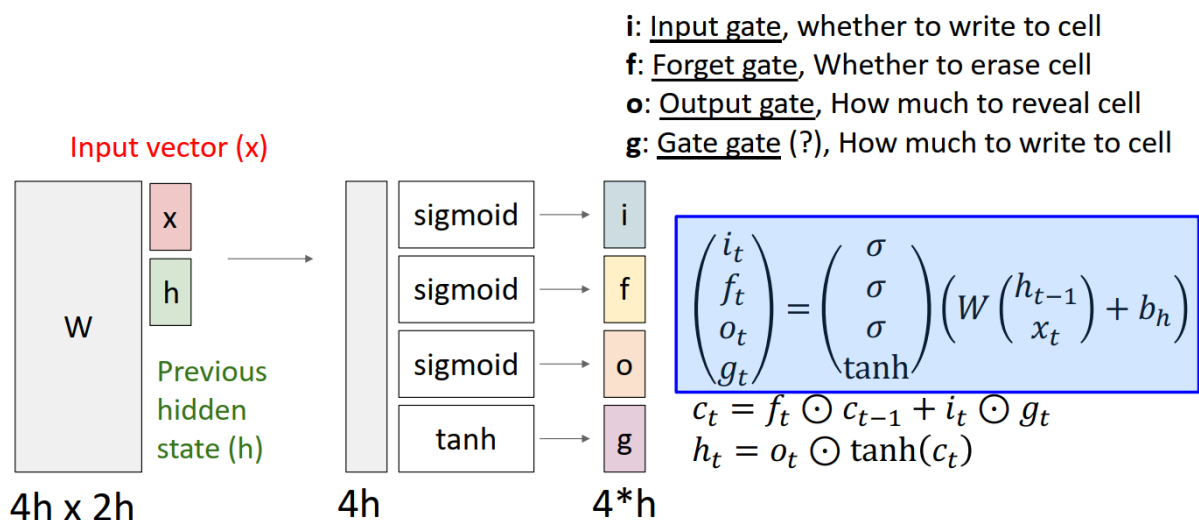
$$\begin{pmatrix} i_t \\ f_t \\ o_t \\ g_t \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} + b \right)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

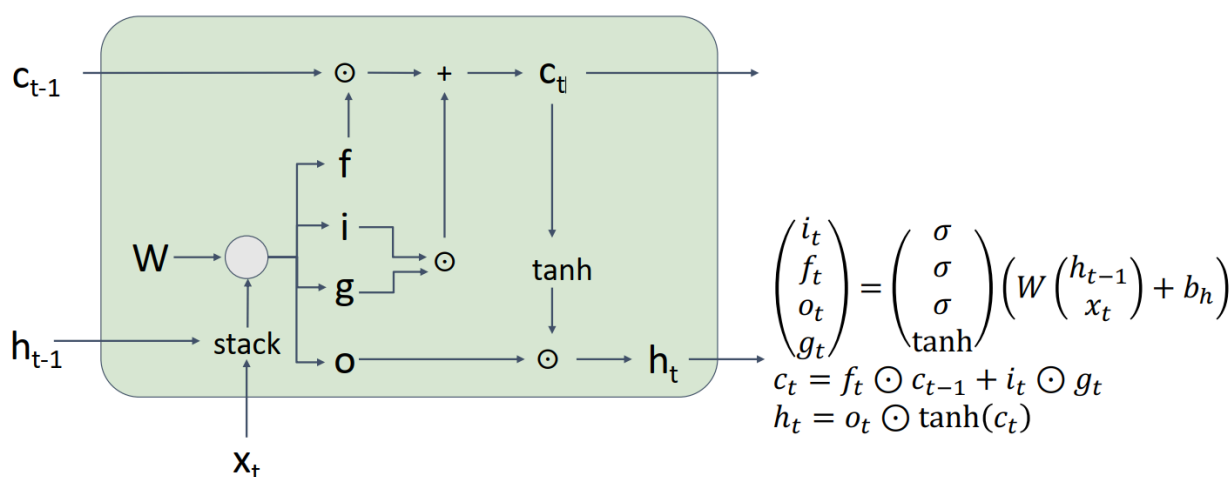
$$h_t = o_t \odot \tanh(c_t)$$

- 第一步：计算四个中间量，分别是输入门 i_t 、遗忘门 f_t 、输出门 o_t 和候选细胞状态 g_t 。
- 第二步：更新细胞状态。用遗忘门 f_t 忘记一部分旧记忆 c_{t-1} ，用输入门 i_t 添加一部分新记忆 g_t 。 $\odot$ 表示逐元素相乘。
- 第三步：计算隐藏状态。将更新后的细胞状态 c_t 通过 **tanh** 压缩，再由输出门 o_t 控制输出。

Long Short Term Memory (LSTM)



Long Short Term Memory (LSTM)



LSTM如何解决梯度消失？ 关键在于细胞状态的更新公式： $c_t = f_t \odot c_{t-1} + \dots$ 。在反向传播时，从 c_t 到 c_{t-1} 的梯度只会逐元素乘以遗忘门 f_t ，没有经过矩阵乘法。这条“高速公路”使得梯度可以几乎无衰减地传递很长的距离，类似于ResNet中的残差连接思想。

需要初始隐藏状态 h_0 和初始单元状态 c_0

6. 其他RNN变体

- **门控循环单元 (Gated Recurrent Unit, GRU)**: 这是LSTM的一个简化版，将遗忘门和输入门合并为一个“更新门”，结构更简单，参数更少，在很多任务上表现与LSTM相当。
- **多层RNN (Multilayer RNN)**: 像堆叠CNN层一样，我们也可以堆叠RNN层。第一层的输出序列作为第二层的输入序列，可以学习到更复杂的特征。

总结

- RNN是为处理**序列数据**而设计的神经网络，其核心是拥有一个可以传递信息的**内部状态（记忆）**。
- RNN非常灵活，可以实现**一对多、多对一、多对多**等多种架构。
- 基础的RNN存在**梯度消失/爆炸**的问题，难以学习长期依赖。
- **LSTM**通过引入**细胞状态和门控机制**（遗忘门、输入门、输出门），有效地解决了梯度消失问题，是目前最主流的RNN变体之一。
- 梯度爆炸可以通过**梯度裁剪**来缓解。

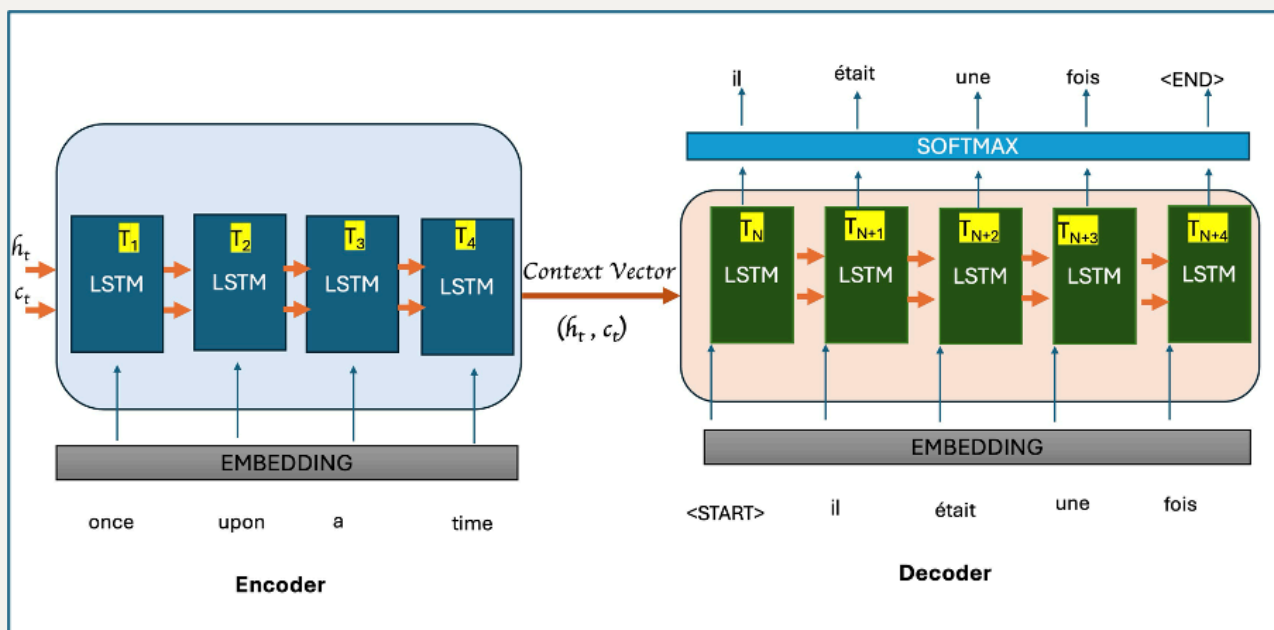
RNN及其变体是自然语言处理、语音识别等领域的基础，理解它们的工作原理是迈向更高级模型（如注意力机制、Transformer）的重要一步。

Encoder-Decoder

- **Encoder (编码器)**: 读取整个输入序列（如一句英文），并将其所有信息压缩成一个固定长度的上下文向量 (**Context Vector, C**)。
- **Decoder (解码器)**: 接收这个上下文向量C，并据此生成输出序列（如一句法文）。

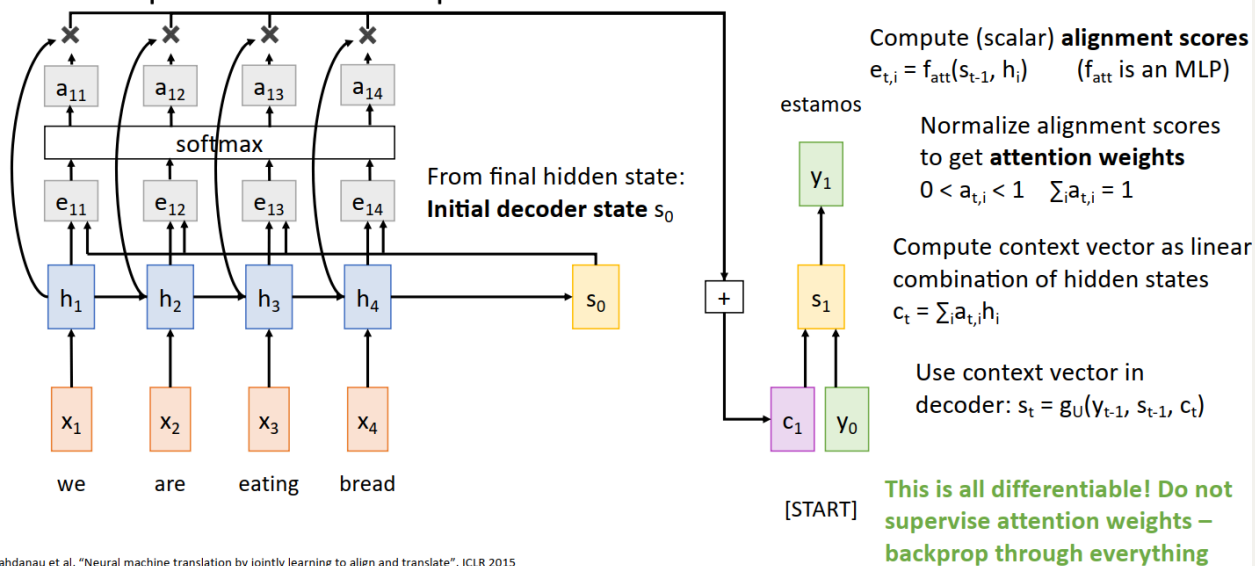
这个架构存在一个严重的问题，被称为“信息瓶颈” (**Information Bottleneck**)。

想象一下，无论输入的句子是5个词还是50个词，编码器都必须将所有含义硬塞进一个同样大小的向量C里。对于长句子来说，这个小小的向量很难记住全部的细节，这极大地限制了模型的性能。



Attention

Sequence-to-Sequence with RNNs and Attention



Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Justin Johnson

Lecture 17 - 15

March 21, 2022

注意力计算

现在，解码器准备生成第一个词。它不再像旧模型那样仅仅依赖一个固定的上下文向量 c ，而是要动态地计算一个专门为这一步服务的上下文向量 c_1 。

这个计算过程分为三步：

• 第一步：计算对齐分数 (Alignment Scores)

- $e_{t,i} = f_{att}(s_{t-1}, h_i)$
- s_{t-1} : decoder的状态, query vector
- 模型会用 s_0 (代表“我接下来要生成什么？”的**Query**) 与编码器所有的隐藏状态 h_1, h_2, h_3, h_4 (代表“输入里都有哪些内容？”的**Keys**) 逐一进行比较。
- 这个比较是通过一个小型的神经网络 f_{att} 完成的，它会为每一对 (s_0, h_i) 打出一个分数 $e_{1,i}$ 。这个分数衡量了输入词 x_i 对于生成下一个输出词的相关性。例如， e_{11} 是 s_0 和 h_1 的分数， e_{12} 是 s_0 和 h_2 的分数，以此类推。

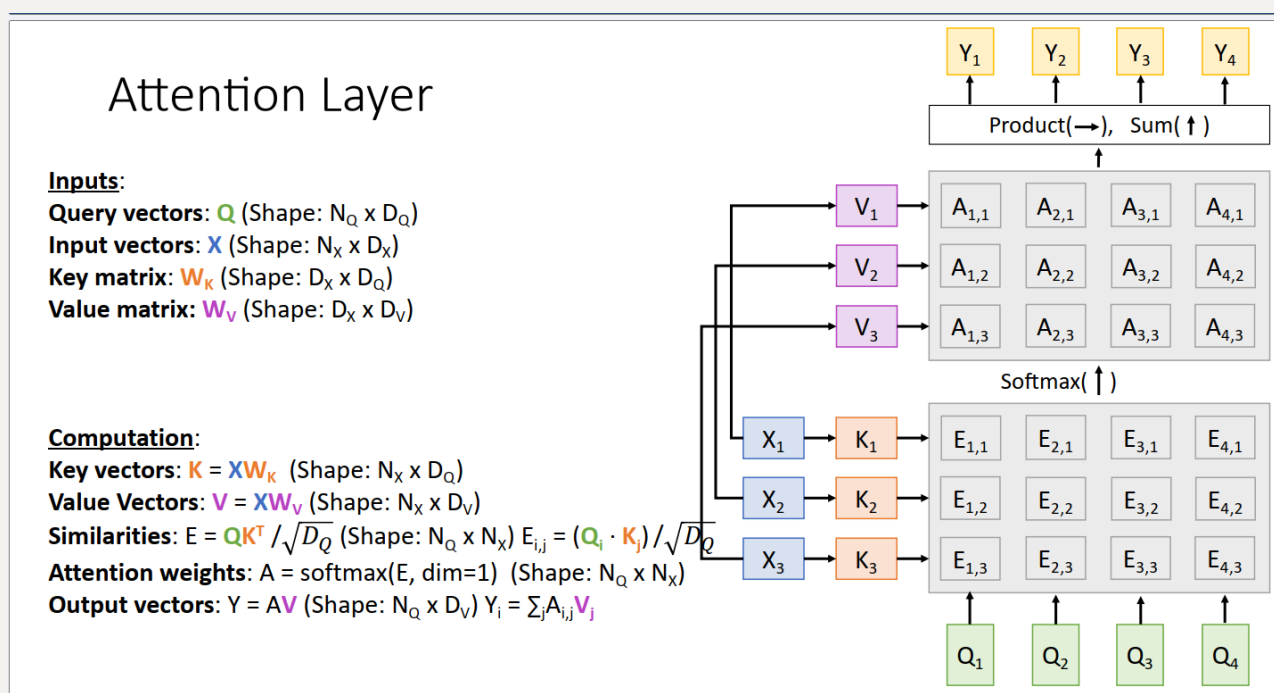
• 第二步：归一化得到注意力权重 (Attention Weights)

- 将刚刚得到的所有对齐分数 (e_{11} 到 e_{14}) 一起放入一个 **softmax** 函数中。
- **softmax** 会将这些分数转换成一组总和为1的概率值，即 $a_{11}, a_{12}, a_{13}, a_{14}$ 。这些就是**注意力权重**。
- $a_{i,:} = \text{softmax}(e_{t,:})$

- 例如，如果模型认为生成第一个词 "estamos" 主要需要关注 "we" 和 "are", 那么 a_{11} 和 a_{12} 的值就会比较大, 而 a_{13} 和 a_{14} 的值就会很小。
- **第三步：计算Context Vector**
 - 公式为 $c_t = \sum a_{t,i} * h_i$ 。
 - 用上一步得到的注意力权重 $a_{1,i}$ 作为“权重”，对编码器的所有隐藏状态 h_i (作为**Values**) 进行加权求和。
 - 这就得到了第一个时间步的上下文向量 c_1 。这个 c_1 是一个为生成 "estamos" 量身定制的上下文向量，它融合了所有输入信息，但重点关注了最相关的部分

但是无法并行计算

Attention layer



一、核心组件与输入定义

图中定义了注意力层的 5 个核心输入：

符号	名称	维度形状	含义解释
Q	Query Vector	$N_Q \times D_Q$	代表当前需要关注的 "问题"
X	输入向量	$N_X \times D_X$	原始输入数据（如序列、图像特征）
W_K	Key Matrix	$D_X \times D_Q$	用于生成键向量的可学习权重
W_V	Value Matrix	$D_X \times D_V$	用于生成值向量的可学习权重

符号	名称	维度形状	含义解释
D_Q, D_V	维度参数	-	查询和值的特征维度

二、Computation

注意力层的计算可分为 **键值生成** → **相似度计算** → **权重归一化** → **加权输出** 四大阶段，对应图中的计算流程：

1. 生成键向量 (Key Vectors)

公式: $K = XW_K$

- 输入: 原始输入 X (维度 $N_X \times D_X$) + 键矩阵 W_K (维度 $D_X \times D_Q$)
- 输出: 键向量 K (维度 $N_X \times D_Q$)
- 作用: 将原始输入转换为与查询向量维度匹配的 "键", 用于后续相似度计算。

2. 生成值向量 (Value Vectors)

公式: $V = XW_V$

- 输入: 原始输入 X (维度 $N_X \times D_X$) + 值矩阵 W_V (维度 $D_X \times D_V$)
- 输出: 值向量 V (维度 $N_X \times D_V$)
- 作用: 将原始输入转换为 "值", 这些值会根据注意力权重进行加权求和。

3. 计算相似度 (Similarities)

公式: $E = \frac{QK^T}{\sqrt{D_Q}}$

- 输入: 查询向量 Q (维度 $N_Q \times D_Q$) + 键向量 K (维度 $N_X \times D_Q$)
- 输出: 相似度矩阵 E (维度 $N_Q \times N_X$)
- 关键细节:
 - QK^T 是查询与键的**点积操作**, 计算每个查询与每个键的相似度
 - $\sqrt{D_Q}$ 是**缩放因子**, 用于避免维度 D_Q 过大导致 softmax 梯度消失

4. 计算注意力权重 (Attention Weights)

公式: $A = \text{softmax}(E, \text{dim} = 1)$

- 输入: 相似度矩阵 E (维度 $N_Q \times N_X$)
- 输出: 注意力权重矩阵 A (维度 $N_Q \times N_X$)
- 作用:
 - softmax 沿维度 1 (N_X 维度) 归一化, 使每行权重和为 1
 - 对每个样本的分数矩阵的最后一个维度 (键的维度) 应用 softmax (若形状为 $N \times K \times K$, 对 $\text{dim}=2$)
 - 权重值越大, 表示对应位置的输入对当前查询越重要

⚠ Warning

A注意力权重 不等于 注意力权重矩阵!!

属性	ATTN_WEIGHTS	W_{attn}
类型	动态计算的权重,是一个中间参数	可学习参数
形状	(N, 4, 4)	(H, 4H)
作用	空间注意力分布	线性变换矩阵
更新	每时间步重新计算	通过梯度下降训练
用途	可视化+加权求和	LSTM门控计算

5. 加权求和生成输出 (Output Vectors)

公式: $Y = AV$

- 输入: 注意力权重 A (维度 $N_Q \times N_X$) + 值向量 V (维度 $N_X \times D_V$)
- 输出: 最终输出 Y (维度 $N_Q \times D_V$)
- 计算逻辑: 对每个查询 Q_i , 输出 $Y_i = \sum_j A_{i,j} \cdot V_j$ 即: 用注意力权重对值向量进行加权求和, 突出重要信息(矩阵乘法, @ / torch.bmm)

右侧的流程图是上述数学公式的具象化演示, 对应 4 个核心步骤:

1. 键值生成 ($K_1/K_2/K_3, V_1/V_2/V_3$)

- $X_1/X_2/X_3$ 通过 W_K 生成 $K_1/K_2/K_3$
- $X_1/X_2/X_3$ 通过 W_V 生成 $V_1/V_2/V_3$

2. 相似度计算 (E 矩阵)

- $Q_1/Q_2/Q_3/Q_4$ 与 $K_1/K_2/K_3$ 计算点积, 生成相似度矩阵 E (如 $E_{1,1} = Q_1 \cdot K_1^T$)

3. 权重归一化 (A 矩阵)

- 对 E 矩阵逐行应用 softmax, 生成注意力权重矩阵 A (如 $A_{1,1}$ 是 $E_{1,1}$ 归一化后的值)

4. 加权输出 ($Y_1/Y_2/Y_3/Y_4$)

- 每个 Y_i 是 A_i (第 i 行权重) 与 V_j (所有值向量) 的加权和 (如 $Y_1 = A_{1,1}V_1 + A_{1,2}V_2 + A_{1,3}V_3$)

四、核心设计思想

1. **动态权重分配**: 注意力机制让模型根据输入内容**动态决定关注哪些位置**, 而非固定权重 (如 CNN 的卷积核、RNN 的固定窗口)。
2. **长距离依赖捕捉**: 通过 $Q - K$ 点积, 模型可以直接计算任意两个位置的关联 (时间 / 空间复杂度 $O(N^2)$), 解决了 RNN 难以捕捉长距离依赖的问题。
3. **可解释性**: 注意力权重矩阵 A 可直观展示模型关注的重点, 辅助调试和优化 (如 NLP 中查看翻译时关注的原文位置)。

Self-Attention

Self-Attention Layer

One **query** per **input vector**

Inputs:

Input vectors: X (Shape: $N_X \times D_X$)

Key matrix: W_K (Shape: $D_X \times D_Q$)

Value matrix: W_V (Shape: $D_X \times D_V$)

Query matrix: W_Q (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$

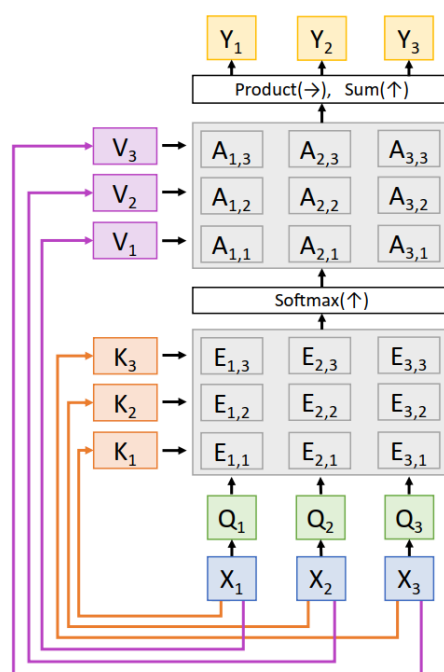
Key vectors: $K = XW_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $V = XW_V$ (Shape: $N_X \times D_V$)

Similarities: $E = QK^T / \sqrt{D_Q}$ (Shape: $N_X \times N_X$) $E_{i,j} = (Q_i \cdot K_j) / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $Y = AV$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Query、Key、Value 均来自同一输入序列，用于挖掘序列内部元素间的关系

步骤：线性变换 → 相似度计算 → 权重归一化 → 加权输出

通过线性变换（三个线性层nn.Linear），将输入 X 映射到三个不同的子空间：

- **Query 向量:** $Q = XW_Q$
 - 输入: $X (N_X \times D_X) + W_Q (D_X \times D_Q)$
 - 输出: $Q (N_X \times D_Q)$
 - 作用: 代表每个输入位置的 "查询", 决定要关注哪些信息

其余与attention相同

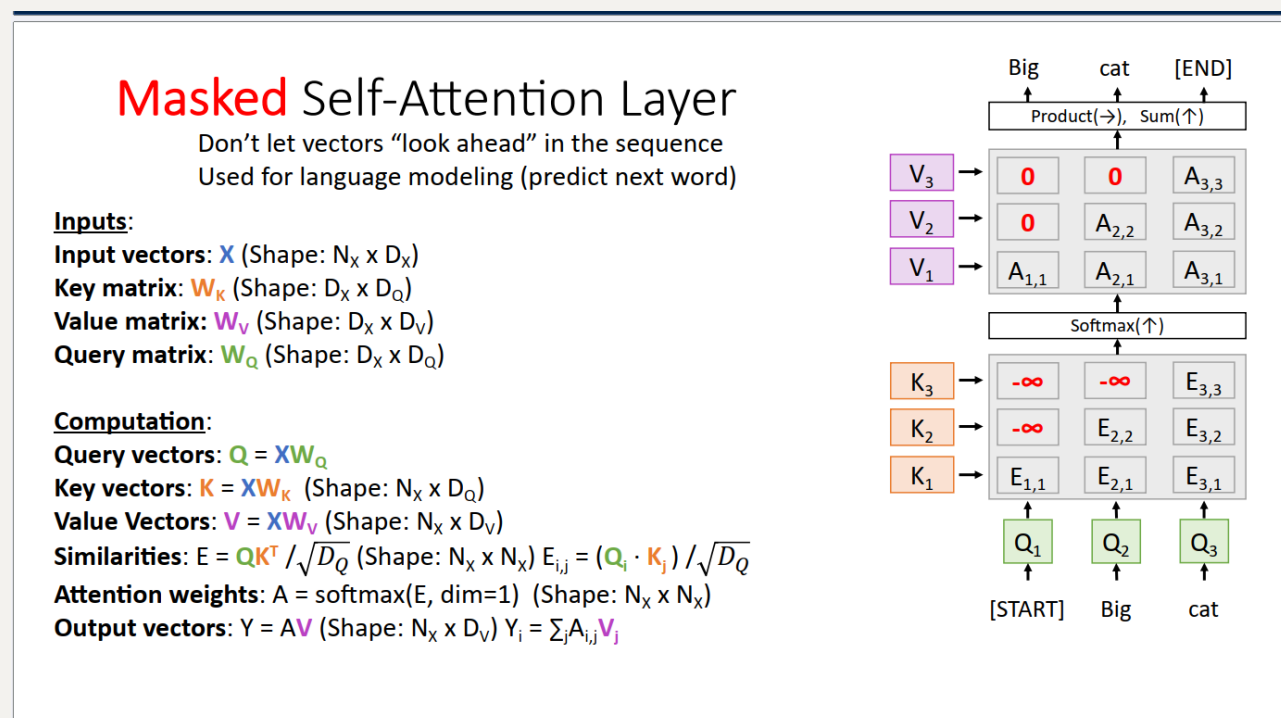
核心设计思想

1. **序列内部关联挖掘:** 每个位置的 Query 都能关注序列中所有位置的 Key, 打破了 RNN/CNN 的固定窗口限制, 直接捕捉长距离依赖。
2. **动态权重分配:** 注意力权重由输入内容动态计算, 而非固定权重 (如 CNN 的卷积核), 让模型自适应关注重要信息。
3. **并行计算优势:** 自注意力的所有步骤 (Q/K/V 生成、相似度计算、加权求和) 都可并行执行, 训练效率远高于 RNN 系列模型。

4. **可解释性**：注意力权重矩阵 A 可直观展示模型关注的重点（如 NLP 中查看翻译时关注的原文位置），辅助模型调试。

不关注输入顺序

Masked Self-Attention



Masked Self-Attention 的作用：

通过掩码（Mask）强制模型在计算注意力时，只能关注“当前位置及之前的词”，模拟人类“根据历史预测未来”的逻辑。

整体计算流程与普通 Self-Attention 一致（生成 Q/K/V、计算相似度、Softmax、加权求和），核心差异在相似度矩阵的掩码处理：

2. 计算相似度矩阵（引入掩码）

普通 Self-Attention 的相似度矩阵 E 是完全可见的，而 Masked Self-Attention 会将“未来位置”的相似度设为极小值（如 $-\infty$ ），让 Softmax 后这些位置的权重趋近于 0。

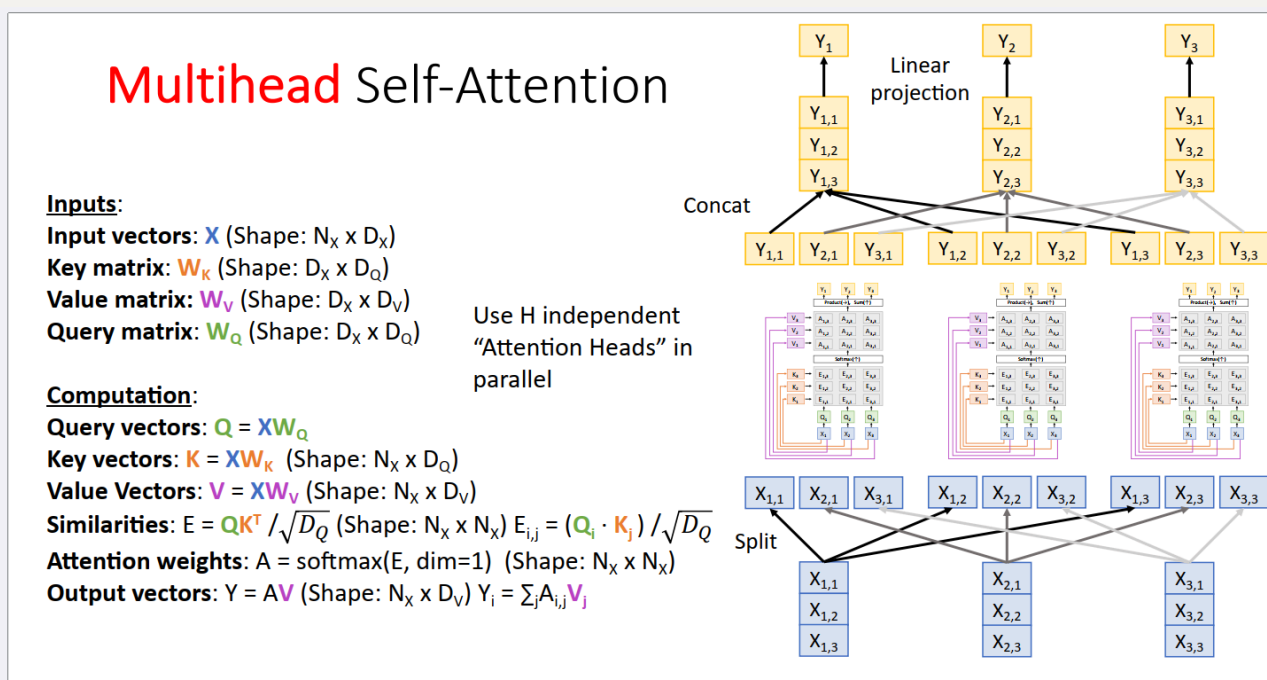
公式（带掩码）：

$$E_{i,j} = \begin{cases} \frac{Q_i \cdot K_j^T}{\sqrt{D_Q}}, & \text{如果 } j \leq i \text{ (当前及历史位置)} \\ -\infty, & \text{如果 } j > i \text{ (未来位置)} \end{cases}$$

右侧流程图示例（输入序列 [START, Big, cat]，共 3 个位置）：

- 对于 Q_1 （对应 **START**）：只能关注 K_1 （自身），所以 $E_{1,2} = -\infty$ 、 $E_{1,3} = -\infty$
- 对于 Q_2 （对应 **Big**）：可以关注 K_1 （**START**）和 K_2 （**Big**），但 $E_{2,3} = -\infty$
- 对于 Q_3 （对应 **cat**）：可以关注 K_1 、 K_2 、 K_3 （无未来位置）

Multihead Self-Attention



Multihead Self-Attention 通过多个独立的注意力头（heads），让模型从不同子空间同时关注序列信息，最终融合多视角结果，增强表达能力

整体流程可分为 输入拆分→多头并行计算→结果拼接→线性投影 四个阶段

1. 输入与基础变换（同普通 Self-Attention）

2. 拆分多头（Split）

将 Q, K, V 按维度拆分为 H 个独立的“头”（图中 $H = 3$ ）：

- 每个头的维度: $D'_Q = D_Q/H$, $D'_V = D_V/H$
- 拆分后形状: Q_h, K_h, V_h 的维度为 $N_X \times D'_Q$ ($h = 1, 2, \dots, H$)
: `emb_dim // num_heads`

3. 多头并行计算注意力（独立计算）

对每个头，独立执行普通 Self-Attention 的计算：

$$\text{相似度} : E_h = \frac{Q_h K_h^T}{\sqrt{D'_Q}} \quad (\text{维度: } N_X \times N_X)$$

$$\text{权重} : A_h = \text{softmax}(E_h, \text{dim} = 1) \quad (\text{维度: } N_X \times N_X)$$

$$\text{输出} : Y_h = A_h V_h \quad (\text{维度: } N_X \times D'_V)$$

关键：每个头独立学习不同的注意力模式（如有的头关注语法结构，有的关注语义关联）

4. 拼接多头结果（Concat）

将所有头的输出 $Y_1, Y_2, \dots, Y_H$ 按维度拼接：

$$Y_{\text{concat}} = \text{Concat}(Y_1, Y_2, \dots, Y_H)$$

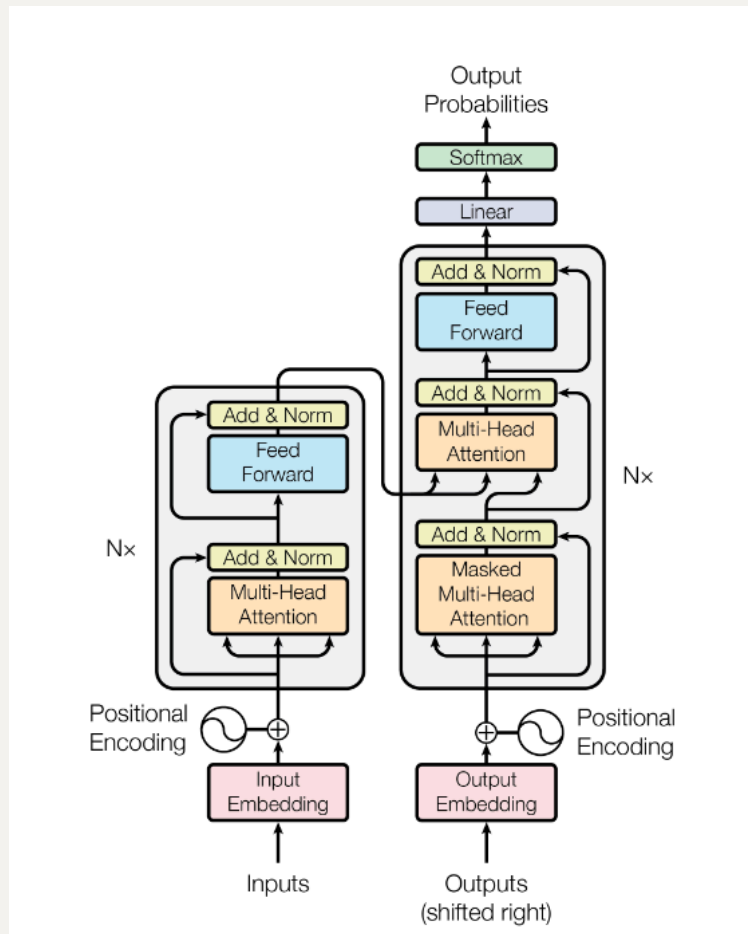
$$\text{拼接后维度: } N_X \times (H \times D'_V) = N_X \times D_V \quad (\text{恢复原始维度})$$

5. 线性投影（Linear Projection）

对拼接后的结果进行线性变换，得到最终输出 Y: $Y = Y_{\text{concat}} W_O$

其中 W_O 是可学习的投影矩阵，确保多头结果有效融合

Transformer



cross attention

两个输入，K，V来自同一个，Q是另一个

Layer Normalization

对单个样本在所有特征维度上进行归一化，使该样本的特征分布满足均值为 0、方差为 1 的标准正态分布，再通过可学习的参数进行缩放和平移，保留特征表达能力。

具体公式如下：

$x.shape: (N, K, M)$, N : batch size; K :sequence length

M :the sequence length embedding特征维度

1. 计算均值和方差：对单个样本的所有特征维度计算均值 (μ) 和方差 (σ^2)

$$\mu = \frac{1}{H} \sum_{i=1}^H x_i$$

$$\sigma^2 = \frac{1}{H} \sum_{i=1}^H (x_i - \mu)^2$$

其中， H 是特征维度数， x_i 是样本在第 i 维的特征值。

2. 归一化：用均值和方差标准化特征

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

其中， ϵ 是一个极小值（如 $1e-5$ ），防止分母为 0。

3. 缩放和平移：通过可学习参数 γ （缩放）和 β （平移）调整归一化后的分布

$$y_i = \gamma \hat{x}_i + \beta$$

β, γ 形状均为 M

```
1 self.scale = nn.Parameter(torch.ones(emb_dim))
2 self.shift = nn.Parameter(torch.zeros(emb_dim))
```

特性	LAYER NORMALIZATION	BATCH NORMALIZATION
归一化范围	单个样本的所有特征维度	批次中所有样本的同一特征维度
依赖批次大小	不依赖（单个样本即可计算）	依赖（批次越小，统计量越不稳定）
适用场景	RNN、Transformer 等序列模型	CNN 等计算机视觉模型
训练 / 推理一致性	完全一致（无需保存移动均值 / 方差）	不一致（推理时用训练阶段的移动统计）

Positional Encoding

- **传递位置信息**：告诉模型序列中每个元素的相对对或绝对对位置（如第 1 个词、第 5 个词）。
- **建模位置关系**：使模型能学习到位置之间的关联（如“距离近的词关联性更强”）

正弦余弦位置编码

这是最经典的位置编码方式，通过正弦弦和余弦函数生成位置信息，公式如下： 对于序列中位置为 `pos` 的元素，其第 `k` 维的位置编码为：

$$\text{PE}_{(\text{pos}, 2k)} = \sin\left(\frac{\text{pos}}{10000^{2k/d_{\text{model}}}}\right)$$
$$\text{PE}_{(\text{pos}, 2k+1)} = \cos\left(\frac{\text{pos}}{10000^{2k/d_{\text{model}}}}\right)$$

其中：

- `pos` 是元素在序列中的位置（从 0 开始）。
- `k` 是维度索引（0, 1, ..., `d_model/2-1`）。
- `d_model` 是嵌入维度（与词嵌入维度一致）。

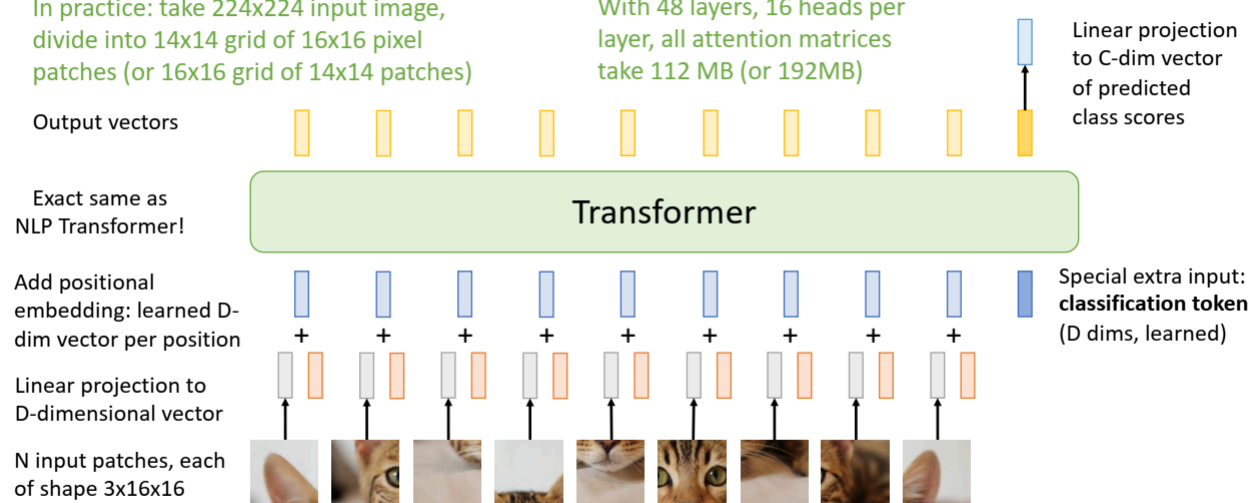
ViT

vision transformer

Vision Transformer (ViT)

In practice: take 224x224 input image,
divide into 14x14 grid of 16x16 pixel
patches (or 16x16 grid of 14x14 patches)

With 48 layers, 16 heads per
layer, all attention matrices
take 112 MB (or 192MB)



Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

Cat image is free for commercial use under a [Pixabay license](#)

Justin Johnson

Lecture 18 - 64

March 23, 2022

1. **图像分块与嵌入**: 将输入图像（例如 224x224 像素）分割成多个小块（例如 16x16 像素）。每个 16x16 的图像块被展平成一个向量。
2. **线性投射**: 将展平后的块向量通过一个线性层，投射到固定的维度（例如 768）。
3. **位置编码**: 为每个块嵌入向量添加一个可学习的位置编码向量，以保留空间信息。
4. **分类令牌 (Classification Token)**: 在序列的开头加入一个特殊的可学习的 “[CLS]”令牌。这个令牌在经过Transformer编码器后的最终输出状态，将被用作整个图像的聚合表示，用于最终的分类任务。
5. **Transformer编码器**: 由多个Transformer块堆叠而成。每个块内部包含多头自注意力层和MLP层，并使用层归一化和残差连接。
6. **分类头**: 将[CLS]令牌的最终输出送入一个MLP头，以预测图像的类别。

drawback: 由于归纳偏置较弱，ViT通常需要在大规模数据集上进行预训练，在较小的数据集上从零开始训练，其性能通常不如ResNet等CNN模型

改进:

1. Augmentation and Regularization :

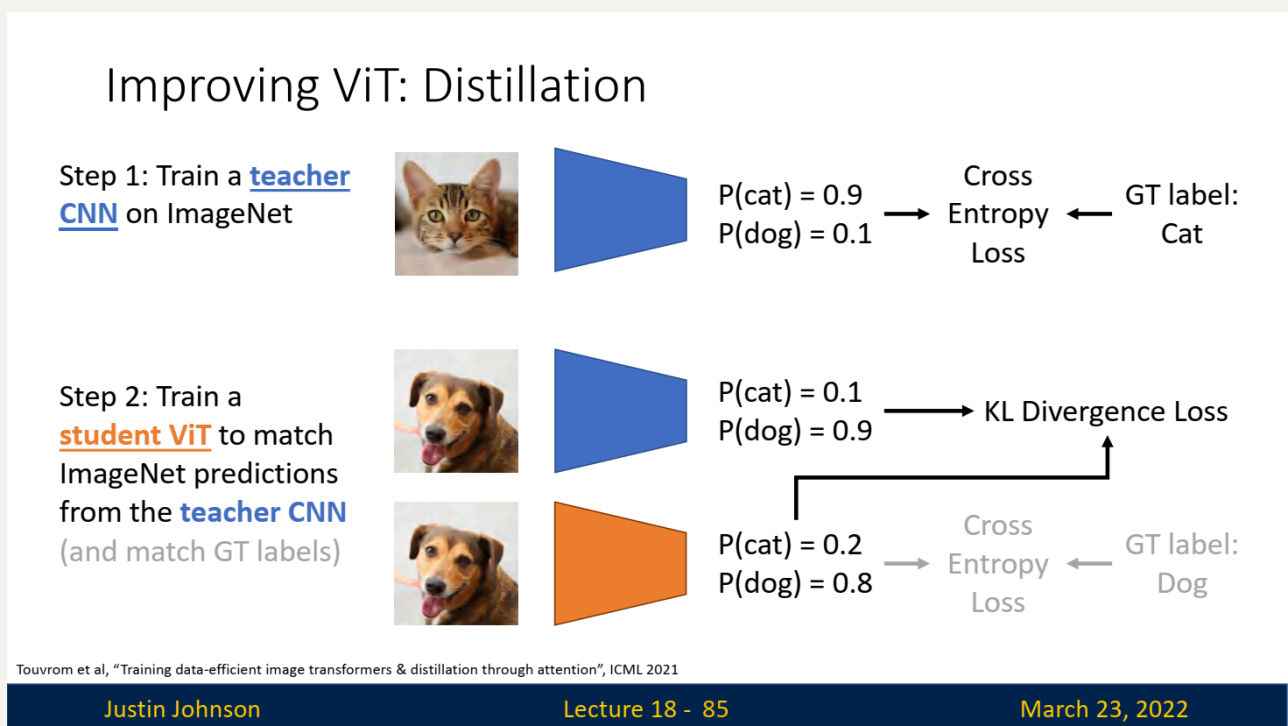
Regularization for ViT models:

- Weight Decay - Stochastic Depth - Dropout (in FFN layers of Transformer)

Data Augmentation for ViT models:

- MixUp
- RandAugment

2.知识蒸馏Distillation



ViT是Isotropic architecture，通道数不变，不进行sample

反之 ResNet等是Hierarchical architecture

Shifted Window Transformer

1.Window-based Multi-Head Self-Attention

不在整张图上做 self-attention，而是把特征图分成很多小窗口（比如 7×7 ），每个窗口里单独做 self-attention；

所以计算复杂度从原来的 $O(N^2)$ 变成 $O(M \cdot k^2)$ ，其中 k 是窗口大小， M 是窗口数。

2.Shifted Window

如果一直在固定窗口里做 attention，不同窗口间就无法交流信息

所以 Swin 的第二个创新是：

在下一层 attention 时，把窗口滑动 (shift) 一半，比如从 $(0,0)$ 开始的窗口，换成从 $(3,3)$ 开始。

- 这样就让不同窗口的信息可以交叉传播；
- 然后再做一次 mask（让 attention 不跨出窗口）。

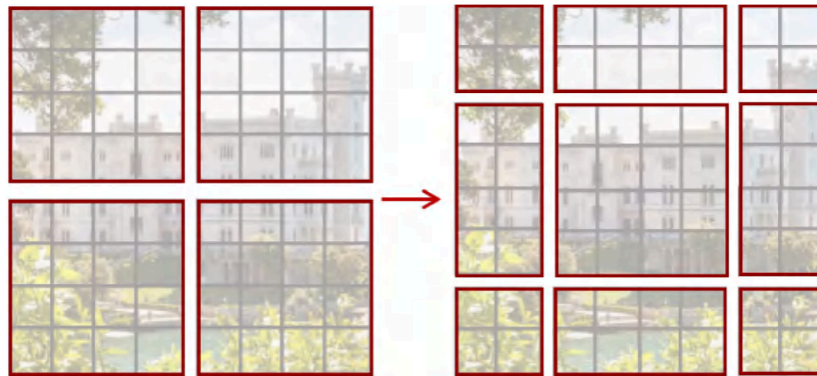
3.Patch Merging（下采样）

在进入下一个 Stage 前，会做类似 CNN 的 downsampling 操作：

- 把相邻的 2×2 patches 合并成一个；
- 通道数加倍，分辨率减半；
- 例如从 $56 \times 56 \rightarrow 28 \times 28$ ，但通道数从 $C \rightarrow 2C$ 。

Swin Transformer: Shifted Window Attention

Solution: Alternate between normal windows and shifted windows in successive Transformer blocks



Block L: Normal windows

Block L+1: Shifted Windows

Liu et al, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows", CVPR 2021

Detail: Relative Positional Bias

ViT adds positional embedding to input tokens, encodes *absolute position* of each token in the image

Swin does not use positional embeddings, instead encodes *relative position* between patches when computing attention:

Attention with relative bias:

$$A = \text{Softmax} \left(\frac{QK^T}{\sqrt{D}} + B \right) V$$

$Q, K, V: M^2 \times D$ (Query, Key, Value)

$B: M^2 \times M^2$ (learned biases)

Justin Johnson

Lecture 18 - 108

March 23, 2022

Swin: 不使用绝对位置嵌入，而是在注意力计算时，通过 **学习相对位置偏置 (B)**，编码 **token** 之间的相对位置关系（如“token A 在 token B 的右上方”）

Object detection

Region-Based CNN

输入图像 → 生成候选区域 → 预处理候选区域 → CNN 特征提取 → SVM 分类 + 边界框回归 → 输出检测结果

边界框回归 (Box Regression)

- **问题:** 候选区域（如 Selective Search 生成的框）可能与真实目标存在偏移（位置、大小偏差），需要修正。
- **方法:** 训练回归器预测边界框的偏移量，公式如下：设候选框坐标为 (x_1, y_1, x_2, y_2) （左上角和右下角），真实框为 $(G_{x1}, G_{y1}, G_{x2}, G_{y2})$ ，回归目标为 4 个偏移量：

$$\begin{cases} t_x = (G_{x1} - x_1)/w \\ t_w = \log(G_{x2} - G_{x1}) - \log(w) \end{cases} \quad \begin{cases} t_y = (G_{y1} - y_1)/h \\ t_h = \log(G_{y2} - G_{y1}) - \log(h) \end{cases}$$

其中, $w = x_2 - x_1$, $h = y_2 - y_1$ 是候选框的宽和高。

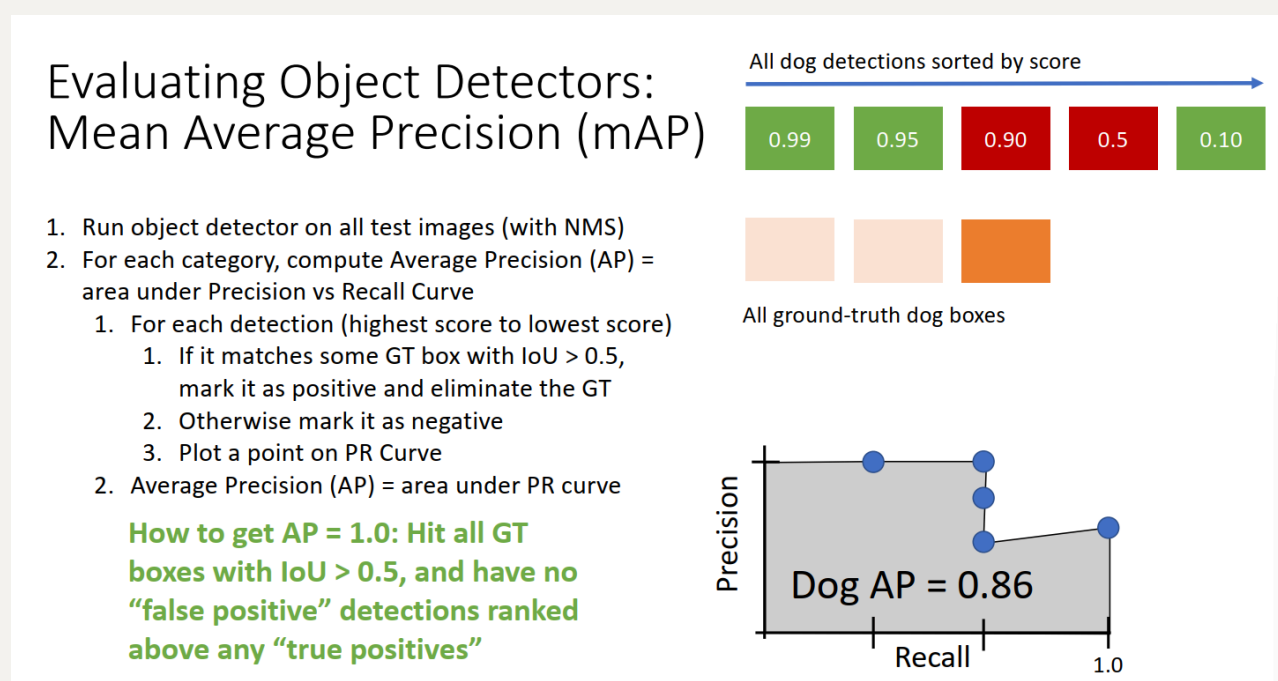
- **回归器**: 以候选区域的 4096 维特征为输入, 预测 (t_x, t_y, t_w, t_h) , 通过线性回归模型 (如 Ridge Regression) 训练

IoU : Intersection over Union = Area of intersection / area of union

Rol : Regions of Interest

后处理: 非极大值抑制 (NMS)

- 经过分类和回归后, 同一目标可能有多个候选框, 需用 NMS 去除冗余框:
 - a. 按分类置信度排序候选框;
 - b. 保留置信度最高的框, 删除与它 IoU 大于阈值 (如 0.5) 的其他框;
 - c. 重复步骤 2 直至所有框处理完毕。



mAP:

- **True Positive (TP, 真正例)**: 模型预测为正样本, 且实际确实是正样本。
- **False Positive (FP, 假正例)**: 模型预测为正样本, 但实际是负样本。
- **False Negative (FN, 假负例)**: 模型预测为负样本, 但实际是正样本。

- **Precision (准确率, P)**: 预测为正样本的结果中, 真正为正样本的比例:

$$P = \frac{TP}{TP+FP}$$

- **Recall (召回率, R)**: 所有实际正样本中, 被模型成功预测为正样本的比例:

$$R = \frac{TP}{TP+FN}$$

AP 的计算逻辑,是 单个类别 的精度 - 召回曲线 (P-R 曲线) 下的面积,

1. **排序**: 将模型对某类别的所有预测结果按**置信度从高到低排序**
2. 对每个检测框, 与真实标注框 (Ground Truth) 对比:
 - 若检测框与某 GT 框的 **IoU (交并比) > 0.5** → 标记为 **True Positive (TP, 真正例)**, 并“消耗”该 GT (避免重复匹配)。
 - 若 $\text{IoU} \leq 0.5$ → 标记为 **False Positive (FP, 假正例)**。
3. 每处理一个检测框, 计算当前的 **Precision (精度)** 和 **Recall (召回率)**, 并在 P-R 曲线中记录点

Fast RCNN

输入图像 → 卷积层提取全局特征图 → 候选区域 (RoI) 生成 → RoI Pooling 层 (固定特征尺寸) → 全连接层 → 并行输出 (分类概率 + 边界框偏移量)

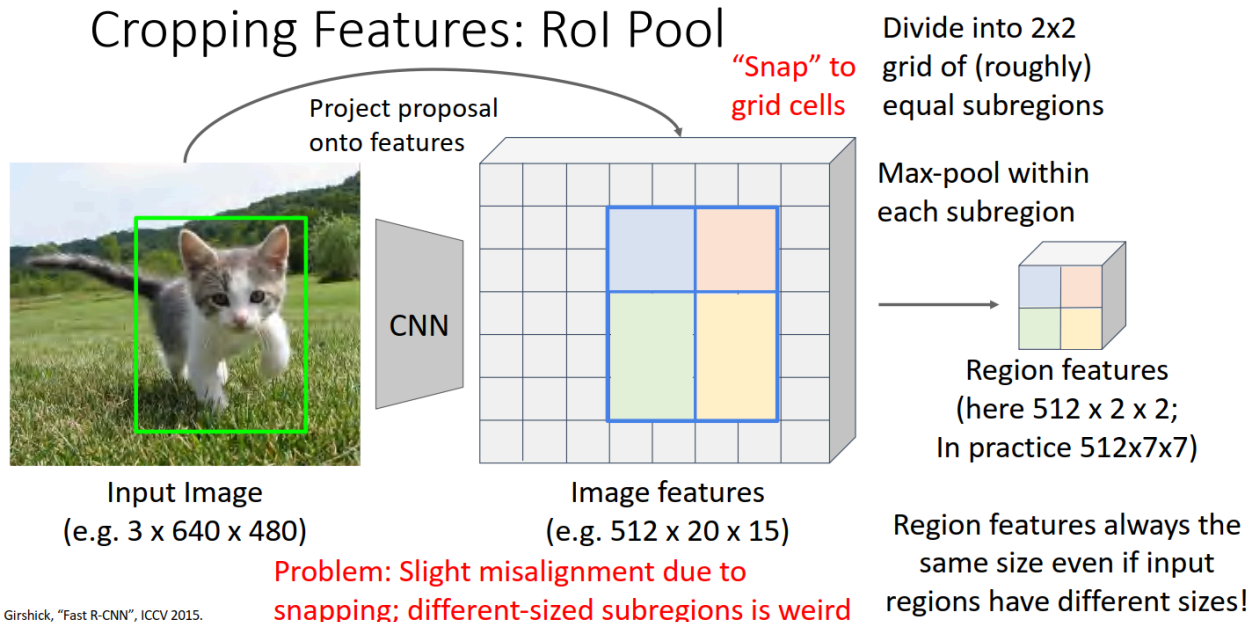
创新:

1. **整图共享卷积特征**:

不再对每个候选区域单独提取特征, 而是先对**整幅图像**做一次卷积操作, 得到全局特征图 (Feature Map), 再从特征图上“裁剪”出候选区域对应的特征 (RoI 特征)。这一改进彻底消除了卷积层的重复计算。

RoI Pooling

Cropping Features: RoI Pool



Justin Johnson

Lecture 14 - 38

March 9, 2022

2. RoI Pooling 层:

候选区域的大小和形状各不相同，对应的特征图区域尺寸也不同。

划分网格：将特征图上的 RoI 区域（尺寸为 $H \times W$ ）划分为 $k \times k$ 个网格（如 7×7 ， k 为超参数），每个网格尺寸为 $(H/k) \times (W/k)$ （允许非整数，向下取整）。

池化操作：对每个网格执行 **max pooling**，得到 1 个值，最终输出 $k \times k$ 的固定尺寸特征图（如 7×7 ）。

- 1 ****示例**：**若 RoI 在特征图上的尺寸为 14×14 ，划分为 7×7 网格，则每个网格 2×2 ，对每个 2×2 网格取最大值，输出 7×7 特征。
- 2
- 3 - 作用：将任意尺寸的 RoI 特征转换为固定尺寸，确保后续全连接层能统一处理。

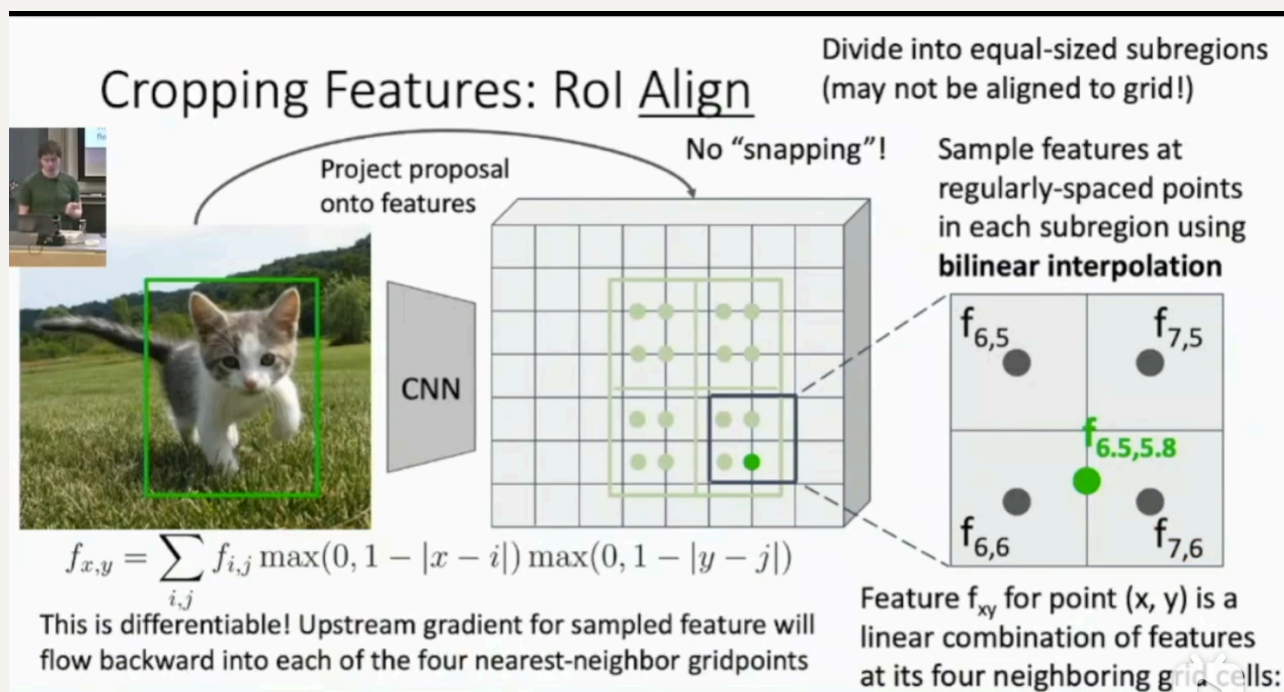
RoI Align

从特征图中提取候选区域（RoI）特征

步骤：

- **保持浮点数坐标：**不取整。
- **均匀分割：**将这个浮点数坐标的区域，精确地分割成目标数量的子网格（bins），例如 7×7 。每个子网格的边界也都是浮点数。

- **规则采样**：在每个子网格中，固定采样 N 个（例如4个）规则分布的点。这些采样点的坐标也都是浮点数。
- **双线性插值 (Bilinear Interpolation)**：将 RoI 划分为固定数量的子区域，对于每个子区域，确定若干个采样点（常见 4 个）。这些采样点的位置由子区域的中心点决定，且不进行取整操作。
- **池化**：最后，对每个子网格中所有采样点的插值结果进行最大池化（或平均池化），得到这个子网格的最终输出值。



3. 多任务损失函数：

将“分类”和“边界框回归”两个任务合并为一个损失函数，实现端到端训练（无需单独训练 SVM 和回归器），大幅简化流程。

Faster RCNN (Two-Stage)

输入: (B, 3, 224, 224)

↓

骨干网络: {"p3": (B,64,28,28), "p4": (B,64,14,14), "p5": (B,64,7,7)}

↓

RPN: 生成~2000个proposals → NMS后保留~300个

↓

ROI Align: 每个proposal → (64, 7, 7)特征

↓

分类器: $(64 \times 7 \times 7) \rightarrow \text{Flatten} \rightarrow \text{Linear} \rightarrow (C+1)$ 类概率

↓

后处理: 置信度过滤 + 类别特定NMS

↓

输出: 最终检测框 + 类别 + 分数

Region Proposal Network(RPN)代替 selective search

RPN 以卷积特征图为输入，包含两个并行分支：

1. **分类分支 (Objectness Score)**: 预测每个位置的锚点“是否包含目标”(二分类: 前景 / 背景)。
2. **回归分支 (BBBox Regression)**: 预测每个锚点到真实目标框的偏移量，修正锚点位置。

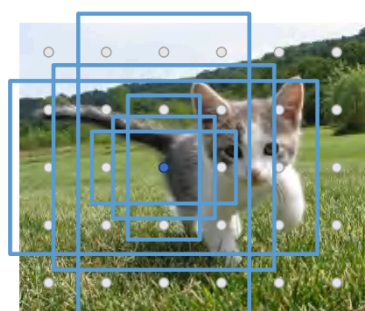
关键概念: **锚点框 (Anchor Boxes)**

为了能够预测不同尺寸和长宽比的物体，RPN 引入了**锚点框 (Anchors)**

- **what?** 锚点框是在特征图的**每一个位置**预先定义好的一组大小和长宽比各不相同的“参考框”或“初始猜测框”。
- **why?** 与其盲目地预测各种可能的边界框，不如从一组固定的、有代表性的初始框开始，然后学习如何对这些初始框进行微调。这大大简化了问题。
- **how?** 例如，在特征图的每一个像素点上，我们都放置 **k** 个锚点框。一个常见的设置是 **k=9**，即3种不同的尺寸（比如 128x128, 256x256, 512x512）乘以3种不同的长宽比（比如 1:1, 1:2, 2:1）。

Region Proposal Network (RPN)

Run backbone CNN to get features aligned to input image



Input Image
(e.g. 3 x 640 x 480)

Each feature corresponds to a point in the input

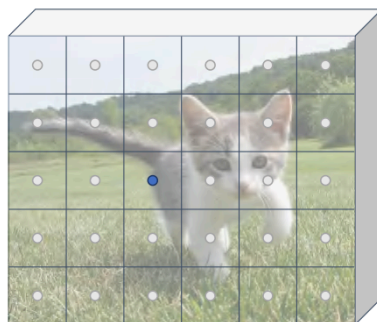
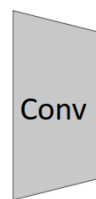


Image features
(e.g. 512 x 5 x 6)

In practice: Rather than using one anchor per point, instead consider K different anchors with different size and scale (here $K = 6$)



Anchor is object?
 $2K \times 5 \times 6$

Anchor transforms
 $4K \times 5 \times 6$

At test-time, sort all $K \times 5 \times 6$ boxes by their positive score, take top 300 as our region proposals

Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015

Feature Pyramid Network

解决问题：图像中的物体有大有小，而传统的 CNN 特征图在深层时分辨率很低，不利于检测小物体

核心思想：利用 CNN 本身就具有的多尺度、金字塔式的特征层级。

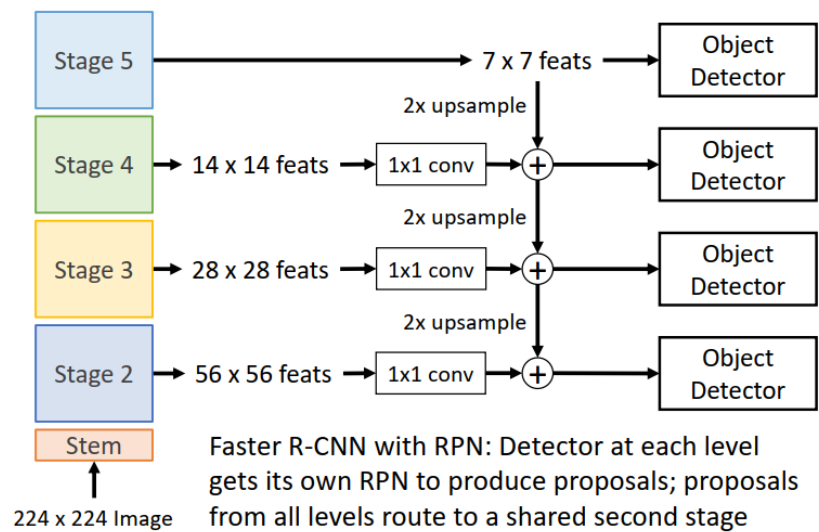
做法：通过一条自顶向下 (**top-down**) 的通路，将高层、语义信息强的特征图进行上采样，并与低层、分辨率高但语义信息弱的特征图进行**横向连接 (lateral connection)**。

效果：最终生成了一系列在所有尺度上都具有丰富语义信息的特征图金字塔。可以在金字塔的每一层上独立进行物体检测，从而高效地处理不同尺寸的物体

Dealing with Scale: Feature Pyramid Network

Add *top down* connections that feed information from high level features back down to lower level features

Efficient multiscale features where all levels benefit from the whole backbone! Widely used in practice



Lin et al, "Feature Pyramid Networks for Object Detection", ICCV 2017

One-Stage :

Retina Net

单阶段目标检测中，存在严重的正负样本不均衡问题。大量易分类的负样本导致检测精度下降

Cross - Entropy Loss : $CE(p_t) = -\log(p_t)$

Solution:

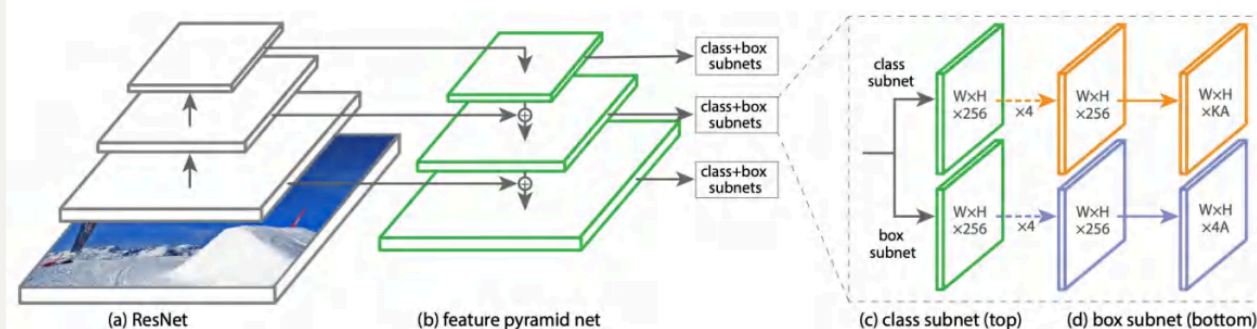
Focal Loss : $FL(p_t) = -(1 - p_t)^\gamma * \log(p_t)$

通过 $(1 - p_t)^\gamma$ 动态调整样本权重，解决类别不平衡问题

同时使用 Feature Pyramid Network

Single-Stage Detectors: RetinaNet

In practice, RetinaNet also uses Feature Pyramid Network to handle multiscale



FCOS

Fully Convolutional One - Stage Object Detection : “Anchor-free” detector

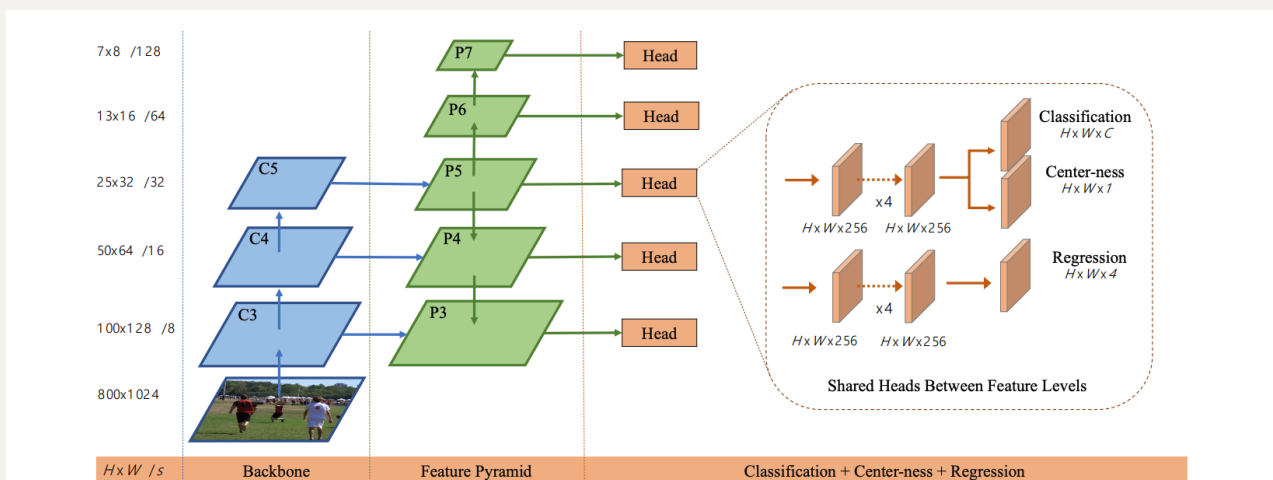


Figure 2 – The network architecture of FCOS, where $C3, C4$, and $C5$ denote the feature maps of the backbone network and $P3$ to $P7$ are the feature levels used for the final prediction. $H \times W$ is the height and width of feature maps. ‘ s ’ ($s = 8, 16, \dots, 128$) is the down-sampling ratio of the feature maps at the level to the input image. As an example, all the numbers are computed with an 800×1024 input.

a. 横向连接层 (Lateral Layers)

目的: 主干网络输出的 $c3, c4, c5$ 特征图虽然空间尺寸不同, 但它们的通道数 (channel/depth) 也各不相同。为了在后续步骤中将它们融合 (例如, 通过相加), 我们必须先将它们的通道数统一。

1*1 Conv2d

b. 输出层 (Output Layers)

目的: 在自顶向下 (Top-down) 路径中, 当高层特征 (如 p5) 被上采样并与低层特征 (如 l4) 融合后, 直接使用融合后的特征效果并不理想。融合后应用一个 3x3 卷积 可以平滑特征并减少上采样可能引入的混叠效应 (aliasing effect), 从而生成更优质的最终特征图 p3, p4, p5。

forward: 自底向上, 得到 c3, c4, c5 -> 横向连接 -> 自顶向下与融合, 上采样, p3, p4, p5

```
1 p4_interim = F.interpolate(p5, scale_factor=2, mode="bilinear") +  
  l4  
2 p4 = self.fpn_params["output4"](p4_interim)
```

设像素点 (x, y) 落在某个真实目标框内, 其回归分支预测的到四条边的距离为

l: 到目标框左边的距离 r: 到目标框右边的距离

t: 到目标框上边的距离 b: 到目标框下边的距离

则中心度 centerness 的定义为:

$$c = \sqrt{\frac{\min(l, r)}{\max(l, r)} \times \frac{\min(t, b)}{\max(t, b)}}$$

Semantic Segmentation

label each pixel in the image with a category label

Fully Convolutional Network:

只有卷积层, 问题: Receptive field 线性增长, 计算效率低

Solution:

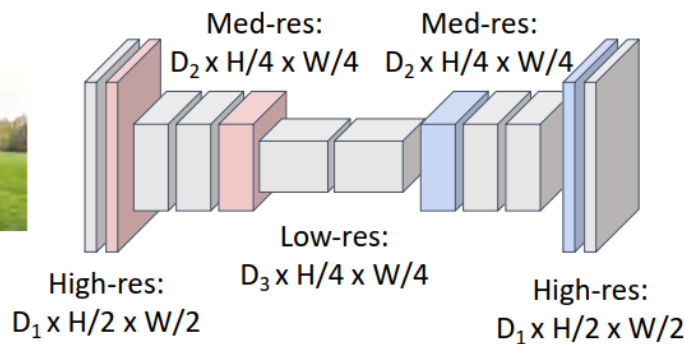
Semantic Segmentation: Fully Convolutional Network

Downsampling:
Pooling, strided
convolution



Input:
 $3 \times H \times W$

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Upsampling:
Interpolation,
transposed conv



Predictions:
 $H \times W$

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Loss function: Per-Pixel cross-entropy

Upsampling

Nearst Neighbor ; Bilinear Interpolation ; Bicubic Interpolation ; Max Unpooling

learnable: **Transposed Convolution** 转置卷积

`nn.ConvTranspose2d`

Learnable Upsampling: Transposed Convolution

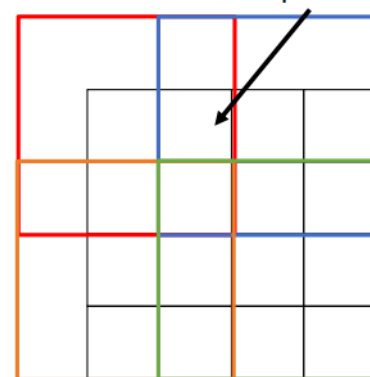
3×3 **convolution transpose**, stride 2

This gives 5×5 output – need to trim one pixel from top and left to give 4×4 output



Input: 2×2

Weight filter by
input value and
copy to output



Output: 4×4

Instance Segmentation

Mask R-CNN:

扩展Faster R-CNN，增加掩码预测分支

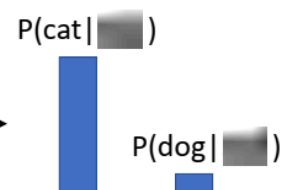
- **掩码预测头：** 它同样以 RoI Align 输出的特征图为输入，通过一系列卷积层（如多层卷积、转置卷积等），为每个候选区域预测一个二进制掩码，该掩码表示目标在图像中的具体像素位置。对于每个类别，都会独立预测一个掩码，掩码的尺寸通常为固定大小（如 28×28 ），后续可以根据实际需求进行上采样，以匹配原始图像的分辨率

Panoptic segmentation

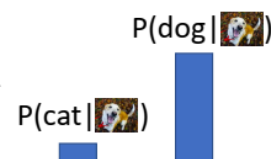
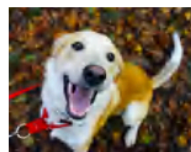
Generative Models

Discriminative vs Generative Models

Discriminative Model:
Learn a probability distribution $p(y|x)$



Generative Model:
Learn a probability distribution $p(x)$



Conditional Generative Model: Learn $p(x|y)$

Discriminative model: the possible labels for each input "compete" for probability mass.
But no competition between **images**

Dog image is CC0 Public Domain

Generative model: All possible images compete with each other for probability mass
Model can "reject" unreasonable inputs by assigning them small values

Taxonomy of Generative Models

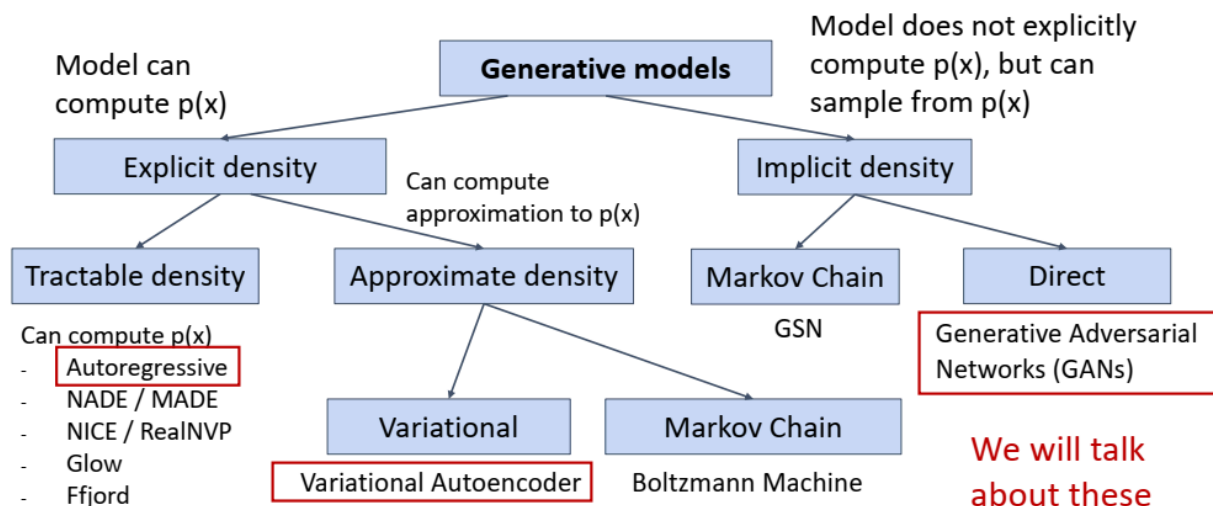


Figure adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017.

Justin Johnson

Lecture 19 - 38

March 28, 2022

维度	显式密度 (EXPLICIT)	隐式密度 (IMPLICIT)
$p(x)$ 计算	能显式计算 (或近似计算)	不计算 $p(x)$, 仅采样
优点	可计算概率 (如异常检测中的概率评分)	采样效率高, 生成质量可能更优 (如 GAN)
缺点	计算成本高 (尤其是高维数据)	理论分析困难 (如 GAN 的收敛性)
典型模型	VAE、自回归模型、流模型	GAN、GSN

Autoregressive Models

$$P(x) = P(x_1) \cdot P(x_2 | x_1) \cdot P(x_3 | x_1, x_2) \cdot \dots \cdot P(x_n | x_1, x_2, \dots, x_{n-1})$$

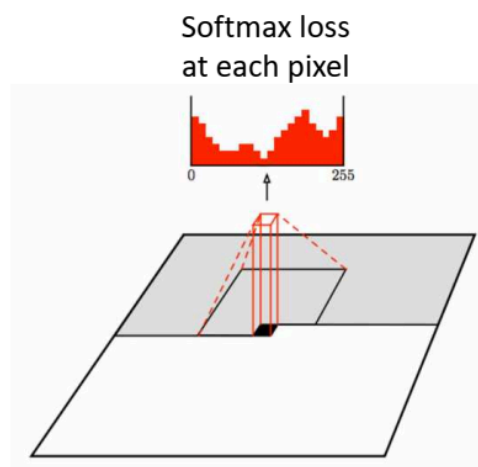
PixelCNN

Still generate image pixels starting from corner

Dependency on previous pixels now modeled using a CNN over context region

Training: maximize likelihood of training images

$$p(x) = \prod_{i=1}^n p(x_i | x_1, \dots, x_{i-1})$$



Van den Oord et al, "Conditional Image Generation with PixelCNN Decoders", NeurIPS 2016

Justin Johnson

Lecture 19 - 59

March 28, 2022

特点	PIXELRNN	PIXELCNN
网络结构	循环神经网络 (RNN)	卷积神经网络 (CNN)
处理顺序	严格顺序处理，一次处理一个像素	并行处理整个图像，使用掩码卷积保证因果关系
计算效率	慢，因为依赖前面像素一步步递归计算	快，因为卷积可以并行计算
长程依赖捕捉	擅长捕捉长距离的依赖	长距离依赖捕捉稍弱，但可通过深层卷积扩展感受野
生成速度	慢（必须一个像素接一个像素生成）	快（推理时仍逐像素生成，但训练和部分计算可并行）

(Regular) Autoencoder

1. Encoder

- 把高维输入（比如一张图像）压缩成一个特征向量

2. Decoder（解码器）

- 从特征向量尝试“还原”原始图像

- 模型的目标是让还原结果尽可能接近输入

没学到输入数据的概率分布，只学了压缩与还原

Variational Autoencoders

Variational Autoencoders

Decoder must be **probabilistic**:
 Decoder inputs z , outputs mean $\mu_{x|z}$
 and (diagonal) covariance $\Sigma_{x|z}$

Sample x from Gaussian with mean
 $\mu_{x|z}$ and (diagonal) covariance $\Sigma_{x|z}$

Sample from
 conditional
 $p_{\theta^*}(x | z^{(i)})$

Sample z
 from prior
 $p_{\theta^*}(z)$

Assume training data $\{x^{(i)}\}_{i=1}^N$ is
 generated from unobserved (latent)
 representation z

Intuition: x is an image, z is latent
 factors used to generate x :
 attributes, orientation, etc.

Assume simple prior $p(z)$, e.g. Gaussian

Represent $p(x|z)$ with a neural network
 (Similar to **decoder** from autencoder)

$$p_{\theta}(\mathbf{x}) = \int_{\mathbf{z}} p_{\theta}(\mathbf{x} | \mathbf{z}) p(\mathbf{z}) d\mathbf{z} \geq \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z} | \mathbf{x})} [\log p_{\theta}(\mathbf{x} | \mathbf{z})] - D_{\text{KL}}(q_{\phi}(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z}))$$

1. **Encoder** 输出的是一个高斯分布：均值 μ 和方差 σ^2

$$q(z | x) = \mathcal{N}(\mu, \sigma^2)$$

2. 采样一个 **latent** 向量 z 来表示输入，然后送入 Decoder 生成图像

直接从分布（如高斯分布）中采样会导致**梯度断裂**。例如，若模型输出均值 μ 和方差 σ^2 ，传统采样 $z \sim \mathcal{N}(\mu, \sigma^2)$ 的操作无法通过反向传播优化 μ 和 σ 。

Reparametrization

通过引入一个与模型无关的**外部噪声变量**

$$z = \mu + \sigma \cdot \epsilon, \text{ 其中 } \epsilon \sim \mathcal{N}(0, 1)$$

Decoder 学的是 $P(x | z)$: 从隐空间还原输入的分

VAE 在 Encoder 端输出的是一个概率分布（高斯），而不是一个确定的点

Loss F:

$$E_{Z \sim q_\phi(z|x)} [\log p_\theta(x|z)] + D_{KL}(q_\phi(z|x), p(z))$$

1. 重建误差 (Reconstruction Loss):

- 希望还原结果 $\hat{x}$ 和原图 x 尽量相似
- 通常是 MSE (图像) 或交叉熵 (文本)

2. KL 散度 (KL Divergence):

- 让 Encoder 输出的分布 靠近标准正态分布 ($\mathcal{N}(0, 1)$)

$$D_{KL}(q_\phi(z|x), p(z)) = -\frac{1}{2} \sum_{j=1}^J (1 + \log(\sigma_{z|x}^2)_j - (\mu_{z|x})_j^2 - (\sigma_{z|x})_j^2)$$

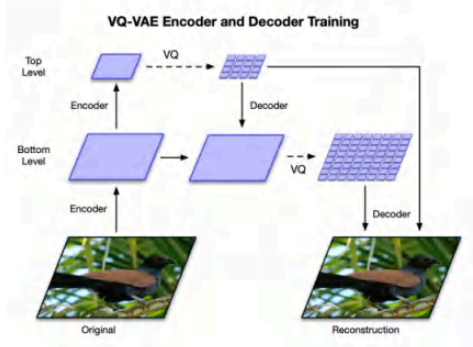
$q_\phi(z|x)$: Z维对角高斯分布, 均值 $\mu_{z|x}$, 标准差 $\sigma_{z|x}$, 形状都是(Z,)

$p(z)$ 是一个零均值、单位方差的Z维高斯分布

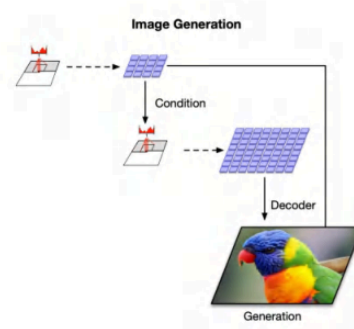
总损失 = 重建损失 + KL 损失

Combining VAE + Autoregressive: Vector-Quantized Variational Autoencoder (VQ-VAE2)

Train a VAE-like model to generate multiscale grids of latent codes



Use a multiscale PixelCNN to sample in latent code space



Razavi et al, "Generating Diverse High-Fidelity Images with VQ-VAE-2", NeurIPS 2019

Generative Adversarial Networks

- 生成器 (G): in: 输入噪声 $\rightarrow$ out: 图像
- $z \sim p(z) \rightarrow G(z)$
- 目标让 $D(G(z)) \approx 1$
- 判别器 (Discriminator): 区分真实与生成图像
- 输入: “真实样本”(来自 p_{data}) 和 “假样本”(来自 G)。
- 输出: 二分类概率 (如 1 表示 “真实”, 0 表示 “fake”), 试图区分二者
- 目标让 $D(x_{\text{real}}) \approx 1$ 且 $D(G(z)) \approx 0$
- 损失函数 (极小极大博弈):

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]$$

The generator loss is: (“目标标签(y=1)”)

$$\ell_G = -\mathbb{E}_{z \sim p(z)} [\log D(G(z))]$$

and the discriminator loss is:

$$\ell_D = -\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] - \mathbb{E}_{z \sim p(z)} [\log (1 - D(G(z)))]$$

无损失曲线

$$V(D, G) = \int_x p_{\text{data}}(x) \log D(x) dx + \int_x p_G(x) \log(1 - D(x)) dx$$

固定 G 后，这变成 D 的函数。

我们对 $D(x)$ 求极值：

$$\text{设 } f(D(x)) = p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x))$$

对 $D(x)$ 求偏导并设为 0：

$$\frac{df}{dD(x)} = \frac{p_{\text{data}}(x)}{D(x)} - \frac{p_G(x)}{1 - D(x)} = 0$$

解得：

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$$

这就是判别器的**最优解**（在给定生成器的前提下）：

▮ 判别器的最佳策略是输出样本来自真实分布的概率。

我们把 $D^*(x)$ 代入整体损失函数，可以推导出：

生成器想**最小化两个分布之间的距离**，即：

$$\min_G V(D^*, G) = \min(-\log(4) + 2 \cdot \text{JS}(p_{\text{data}} \| p_G))$$

其中 JS 是 Jensen-Shannon 散度（两个分布的对称 KL 距离），当且仅当 $p_G = p_{\text{data}}$ 时为 0

Kullback-Leibler Divergence : $\text{KL}(p \| q) = \sum_x p(x) \cdot \log \left(\frac{p(x)}{q(x)} \right)$

Jensen-Shannon Divergence: $JS(p, q) = \frac{1}{2}KL\left(p, \frac{p+q}{2}\right) + \frac{1}{2}KL\left(q, \frac{p+q}{2}\right)$

Least Squares GAN

$$\ell_G = \frac{1}{2} \mathbb{E}_{z \sim p(z)} \left[(D(G(z)) - 1)^2 \right]$$

and the discriminator loss:

$$\ell_D = \frac{1}{2} \mathbb{E}_{x \sim p_{\text{data}}} \left[(D(x) - 1)^2 \right] + \frac{1}{2} \mathbb{E}_{z \sim p(z)} \left[(D(G(z)))^2 \right]$$

In these equations, we assume that the output from the discriminator is an unbounded real number $-\infty < D(x) < \infty$.

LSGAN判别器直接输出scores, 不需要sigmoid

scores = discriminator(x) # 范围: $(-\infty, +\infty)$

不需要转换为概率, 直接用于计算平方损失

conditional GAN

生成器变成:

$$G(z, y) \rightarrow \text{生成带条件的图像 (例如数字 3)}$$

判别器变成:

$$D(x, y) \rightarrow \text{判断图像 } x \text{ 是否是类 } y \text{ 的真图像}$$

Conditional GANs: Conditional Batch Normalization

Batch Normalization

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Learn a separate
scale and shift
for each
different label y

Conditional Batch Normalization

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

$$y_{i,j} = \gamma_j^y \hat{x}_{i,j} + \beta_j^y$$

Dumoulin et al, "A learned representation for artistic style", ICLR 2017

Justin Johnson

Lecture 20 - 106

March 30, 2022

模型 LATENT 是什么

VAE 隐变量 z ，代表数据的压缩抽象

GAN 输入是 latent 向量 $z \sim \mathcal{N}(0, 1)$ ，表示“我想画一个像这样的图”

Diffusion latent 可以是图像特征空间（Stable Diffusion）或高维噪声空间

项目	图像特征 (CNN)	LATENT 表示 (VAE、DIFFUSION等)
来源	卷积层提取的特征图	编码器压缩后的语义表示
空间大小	比原图小一些，比如 $7 \times 7 \times 512$	更小更稠密，比如 $64 \times 64 \times 4$
可视化性	通常不能直接还原成图	可以解码回图像（尤其在 VAE / Stable Diffusion 中）
目的	识别、分类、分割等	生成、编辑、理解图像
信息结构	多是局部特征	更偏向全局语义、抽象表达

自监督学习

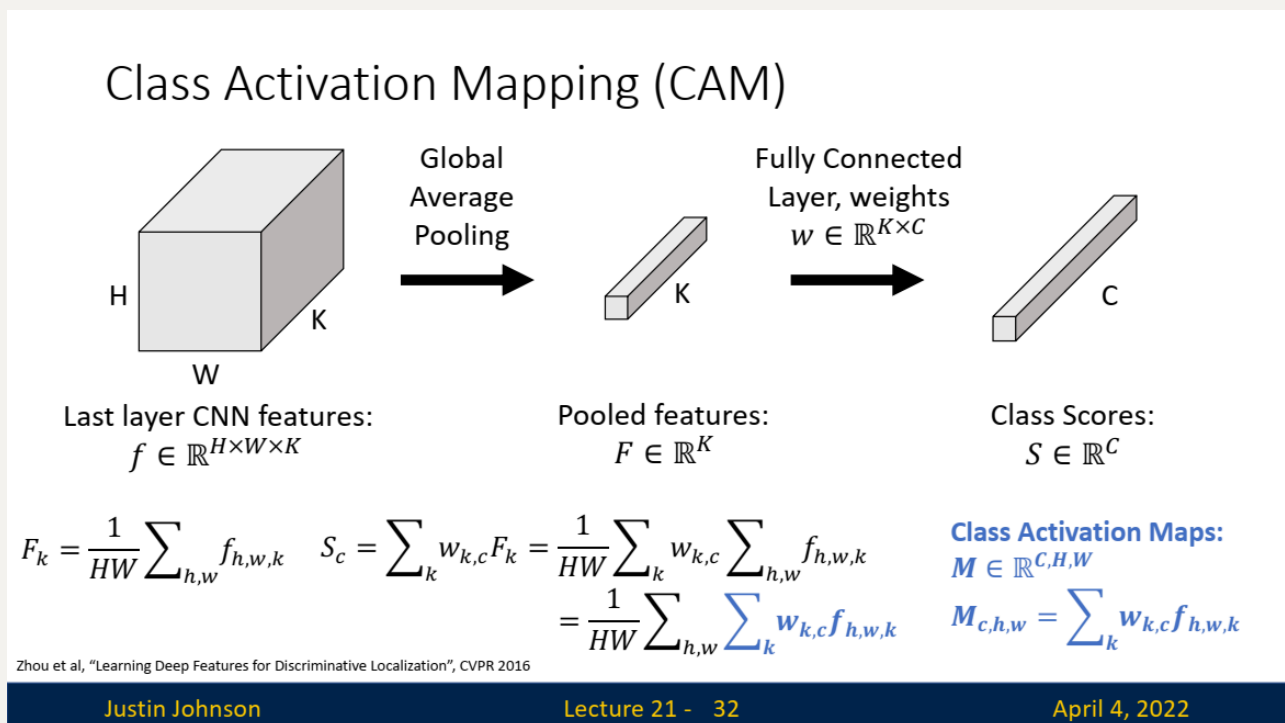
- 对比学习 (SimCLR):

- 正样本对：同一图像的不同增强版本。
- 损失函数：InfoNCE Loss，最大化正样本相似度。

$$L = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)}$$

Visualization

Class Activation Mapping



数学公式：

全局平均池化： $F_k = \frac{1}{HW} \sum_{h,w} f_{h,w,k}$

计算类别分数： $S_c = \sum_k w_{c,k} F_k$

生成类激活映射 (Class Activation Maps):

$$M_{c,h,w} = \sum_k w_{c,k} f_{h,w,k}$$

核心逻辑：将类别 c 的权重 $w_{c,k}$ 与原始特征图 $f_{h,w,k}$ 相乘后累加，得到 空间分辨率为 $H \times W$ 的热力图 M_c

$$S_c = \sum_k w_{c,k} \left(\frac{1}{HW} \sum_{h,w} f_{h,w,k} \right) = \frac{1}{HW} \sum_{h,w} \left(\sum_k w_{c,k} f_{h,w,k} \right) = \frac{1}{HW} \sum_{h,w} M_{c,h,w}$$

结论：类别分数 S_c 本质是 类激活映射 M_c 的全局平均，即模型的分类决策是对热力图中“高贡献区域”的平均响应

Problem: Can only apply to last conv layer !

Gradient-Weighted CAM

Gradient-Weighted Class Activation Mapping (Grad-CAM)

1. Pick any layer, with activations $A \in \mathbb{R}^{H \times W \times K}$
2. Compute gradient of class score S_c with respect to A :

$$\frac{\partial S_c}{\partial A} \in \mathbb{R}^{H \times W \times K}$$

3. Global Average Pool the gradients to get weights $\alpha \in \mathbb{R}^K$:

$$\alpha_k = \frac{1}{HW} \sum_{h,w} \frac{\partial S_c}{\partial A_{h,w,k}}$$

4. Compute activation map $M^c \in \mathbb{R}^{H,W}$:

$$M_{h,w}^c = ReLU \left(\sum_k \alpha_k A_{h,w,k} \right)$$

Selvaraju et al, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization", CVPR 2017

1. 选定一个卷积层， $A \in \mathbb{R}^{H \times W \times K}$
2. 计算类别分数 S_c 对激活 A 的梯度 $\frac{\partial S_c}{\partial A} \in \mathbb{R}^{H \times W \times K}$ ，反传到这个卷积层
3. 对梯度做全局平均池化，得到通道权重 $\alpha \in \mathbb{R}^K$

$$\alpha_k = \frac{1}{HW} \sum_{h,w} \frac{\partial S_c}{\partial A_{h,w,k}}$$

得到一个粗糙的“类激活图”(Class Activation Map)

4. 加权激活并 ReLU, 生成热力图 $M^c \in \mathbb{R}^{H,W}$

$$M_{h,w}^c = \text{ReLU} \left(\sum_k \alpha_k \cdot A_{h,w,k} \right)$$

, 叠加在输入图上作为热力图, **ReLU 过滤**: 只保留 **正贡献区域** (负贡献会降低 S_c , 模型决策更依赖正贡献区域, 因此过滤负梯度对应的激活)

维度	CAM	GRAD-CAM
适用网络	仅支持“GAP + 全连接”的 CNN	任意 CNN (ResNet、DenseNet 等)
层选择	仅最后一层卷积	任意层 (可分析层级特征)
权重来源	全连接层权重	梯度 (更直接反映决策依赖)
负贡献处理	未过滤 (可能包含抑制区域)	ReLU 过滤负贡献, 只显式正区域

Gradient Ascent

Visualizing CNN Features: Gradient Ascent

(Guided) backprop:
Find the part of an image that a neuron responds to

Gradient ascent:
Generate a synthetic image that maximally activates a neuron

$$I^* = \arg \max_I \boxed{f(I)} + \boxed{R(I)}$$

Neuron value Natural image regularizer

优化目标 $I^* = \arg \max_I [f(I) + R(I)]$

1. $f(I)$: 神经元激活值 (Neuron value)

- $f(I)$ 表示 **图像 I 对目标神经元的激活强度**（如某层某通道的输出值）。
- 梯度上升的核心是 **最大化 $f(I)$** ：让合成图像尽可能“刺激”目标神经元，暴露其学习的特征模式。

2. $R(I)$ ：自然图像正则项 (Natural image regularizer)

- 若仅最大化 $f(I)$ ，生成的图像会充满噪声（如像素剧烈震荡），缺乏“自然图像”的规律（如平滑、颜色合理）。
- $R(I)$ 用于 约束图像的“自然性”：
 - 常见形式：L2 正则（限制像素值范围）、总变分（TV，促进平滑）、对抗正则（用 GAN 判别器约束图像真实感）。

3. I^* ：优化后的合成图像

通过 **迭代梯度上升** 求解：

1. 初始化图像 I （如随机噪声）；
2. 计算 $f(I)$ 对 I 的梯度 $\nabla_I f(I)$ ；
3. 沿梯度方向更新 I ($I \leftarrow I + \alpha \nabla_I f(I)$, $\alpha : \text{lr}$)；
4. 同时施加 $R(I)$ 的约束，让图像更自然。

揭示神经元“学到了什么”

- **底层神经元**：生成的图像可能是**边缘、纹理**（如某通道专门响应“水平边缘”）。
- **高层神经元**：生成的图像可能是**复杂语义模式**（如某通道响应“人脸”“汽车”，甚至抽象概念）。
- **示例**：若某高层神经元的梯度上升结果是“带条纹的猫脸”，说明该神经元学习到了“猫脸 + 条纹”的组合特征。

方法	(GUIDED) BACKPROP	GRADIENT ASCENT
目标	定位 已有图像 中让神经元响应的区域	生成 新图像 ，最大化神经元的激活
逻辑	从数据中“挖掘”神经元关注的部分	反向构造“最能激活神经元的输入”

Feature Inversion

Given a CNN feature vector for an image, find a new image that:

- Matches the given feature vector
- "looks natural" (image prior regularization)

$$\mathbf{x}^* = \underset{\mathbf{x} \in \mathbb{R}^{H \times W \times C}}{\operatorname{argmin}} \ell(\Phi(\mathbf{x}), \Phi_0) + \lambda \mathcal{R}(\mathbf{x})$$

Given feature vector

$$\ell(\Phi(\mathbf{x}), \Phi_0) = \|\Phi(\mathbf{x}) - \Phi_0\|^2$$

Features of new image

$$\mathcal{R}_{V^\beta}(\mathbf{x}) = \sum_{i,j} \left((x_{i,j+1} - x_{ij})^2 + (x_{i+1,j} - x_{ij})^2 \right)^{\frac{\beta}{2}}$$

Total Variation regularizer (encourages spatial smoothness)

Mahendran and Vedaldi, "Understanding Deep Image Representations by Inverting Them", CVPR 2015

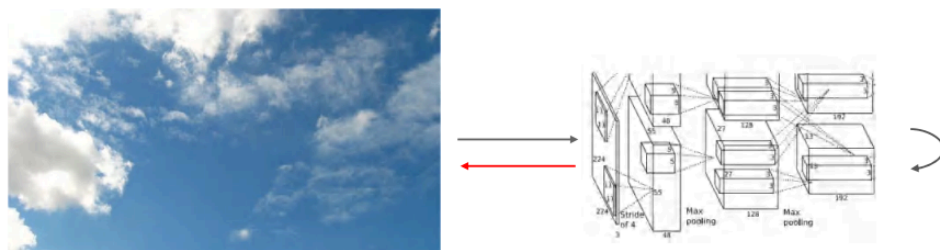
Justin Johnson

Lecture 21 - 60

April 4, 2022

DeepDream: Amplify Existing Features

Rather than synthesizing an image to maximize a specific neuron, instead try to **amplify** the neuron activations at some layer in the network



Choose an image and a layer in a CNN; repeat:

1. Forward: compute activations at chosen layer
2. Set gradient of chosen layer *equal to its activation*
3. Backward: Compute gradient on image
4. Update image

Equivalent to:

$$I^* = \arg \max_I \sum_i f_i(I)^2$$

Mordvintsev, Olah, and Tyka, "Inceptionism: Going Deeper into Neural Networks", [Google Research Blog](https://research.google.com/pubs/pub45329.html). Images are licensed under [CC-BY 4.0](https://creativecommons.org/licenses/by/4.0/)

Justin Johnson

Lecture 21 - 63

April 4, 2022

原图像 x

↓

提取某层特征 $F(x)$

↓

计算 loss = 特征强度

↓

对图像 x 做梯度上升以增强这些特征

↓

反复多次 → 梦幻般的图像

Neural Texture Synthesis

方法一：基于 **Gram Matrix** 的纹理合成（来自 Style Transfer）

由 **Gatys et al. (2015)** 提出，使用 CNN 提取特征图后，用它们的**统计特性**（如协方差）来描述纹理。

步骤：

1. 把纹理图 T 输入预训练 CNN（如 VGG）
2. 提取中间层的特征图 $F_l(T)$
3. 计算 Gram Matrix：

$$G_l = F_l F_l^T$$

描述不同通道之间的共现（即“纹理关系”）

4. 初始化一张随机图像 x ，优化它的像素值，使它的 Gram Matrix 逼近原图的

$$\mathcal{L}_{\text{texture}} = \sum_l \|G_l(x) - G_l(T)\|^2$$

就像是“反向喂图”，让随机图像长出一样的纹理统计特征。

方法二：使用神经网络直接合成纹理（Feedforward Texture Nets）

Gatys的方法慢，要优化图像像素。后来有方法用一个可训练的网络来一次性生成纹理。

特点：

- 使用 U-Net / CNN 学习从噪声 z 到纹理图 x 的映射
- 可实现**实时合成**
- 损失仍然用 Gram Matrix 或感知损失

这种方式可以直接用于：

- 材质合成
- 视频纹理迁移
- GAN中纹理生成模块

方法三：基于生成模型（GAN、Diffusion）

GAN-based Texture Synthesis

- 用 PatchGAN、StyleGAN 等模型，从 latent 向量生成无限大纹理
- 可控制风格、分辨率等因素

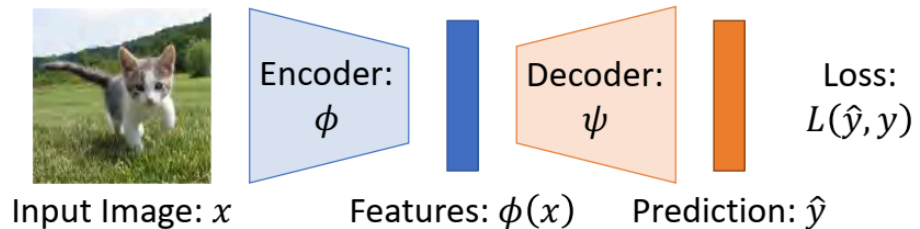
Diffusion-based Texture Generation

- 使用条件扩散模型，让模型逐步生成特定纹理特征的图像
- 可以加入文本提示（“金属感丝绒”）

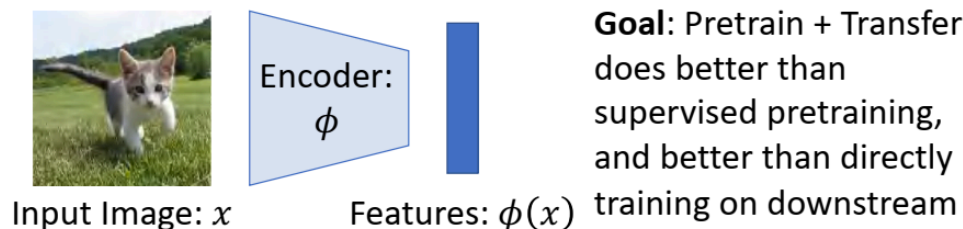
Self-Supervised Learning

Self-Supervised Learning: Pretext then Transfer

Step 1: Pretrain a network on a pretext task that doesn't require supervision



Step 2: Transfer encoder to downstream tasks via linear classifiers, KNN, finetuning



1. Pretext task

从原始数据 x 中，用某种方式加工变形成：

- 输入 $\tilde{x}$ ：加噪、打乱、遮盖、增强
- 伪标签 y ：比如原图、图像旋转角度、增强前图

模型的任务是预测这个“伪标签”。

2. Representation Learning

① 编码器 (Encoder)

模型结构可以是：

- CNN (图像：ResNet、ViT)
- Transformer (语言：BERT、GPT)
- 自编码器 (图像/文本)

得到表示向量 $f(x) \in \mathbb{R}^d$

Fine-tuning / Transfer

◆ 方法一：线性探测器 (Linear Probe)

- 冻结编码器
- 在特征上训练一个线性分类器
- 测试这个表征有多可分

用途：评估表征质量

◆ 方法二：全模型微调 (Fine-tuning)

- 用少量标注数据
- 解冻部分或全部 **encoder**
- 微调模型适应具体任务（如分类、检测、QA）

用途：真实部署任务（比如医学图像分类、语义分割）

对比学习

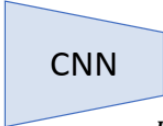
Contrastive Learning

Assume we don't have labels for images, but we know whether some pairs of images are **similar** or **dissimilar**

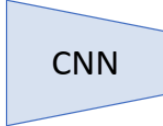
Let $d = \|\phi(x_1) - \phi(x_2)\|_2$ be the Euclidean distance between features for two images

Similar images should have similar features **Dissimilar** images should have dissimilar features

Problem: Where to get positive and negative pairs?


$L_S(x_1, x_2) = d^2$
Pull features together


$L_D(x_1, x_2) = \max(0, m - d)^2$
Push features apart (up to margin m)

Hadsell et al, "Dimensionality Reduction by Learning and Invariant Mapping", CVPR 2006

[White kitten image](#) is free for commercial use under the [Pixabay license](#)

Justin Johnson
Lecture 22 - 78
April 6, 2022

Masked Autoencoders (MAE)

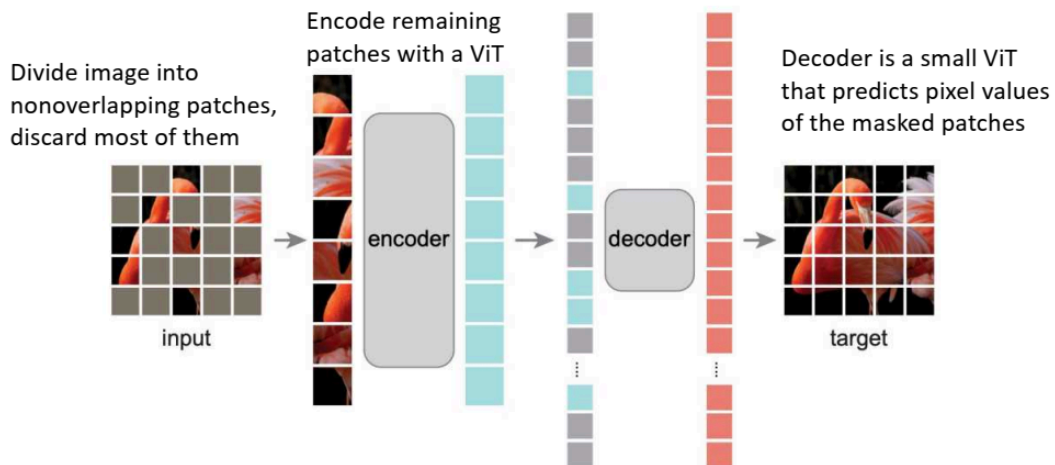
遮盖（mask）输入数据的部分内容

编码器只对剩余未遮盖的部分进行编码

解码器利用编码信息重建完整输入（包括被遮盖的部分）

Masked Autoencoders (MAE)

A ~~new~~ old method dethrones contrastive learning? Denoising Autoencoder with Vision Transformer



He et al, "Masked Autoencoders are Scalable Vision Learners", CVPR 2022

Justin Johnson

Lecture 22 - 93

April 6, 2022

Summary

Self-Supervised Learning (SSL) aims to scale up to larger datasets without human annotation

First train for a pretext task, then transfer to downstream tasks

Many pretext tasks: context prediction, jigsaw, colorization, clustering, rotation
SSL has been wildly successful for language

Intense research on SSL in vision; current best are contrastive, masked autoencoding

Multimodal SSL uses images together with additional context

Multimodal SSL with vision + language has been very successful; seems very promising!

3D Vision

Depth Map

Predicting Depth Maps

Scale invariant loss

$$D(y, y^*) = \frac{1}{2n^2} \sum_{i,j} ((\log y_i - \log y_j) - (\log y_i^* - \log y_j^*))^2$$
$$= \frac{1}{n} \sum_i d_i^2 - \frac{1}{n^2} \sum_{i,j} d_i d_j = \frac{1}{n} \sum_i d_i^2 - \frac{1}{n^2} \left(\sum_i d_i \right)^2$$

RGB Input Image:
3 x H x W

Fully Convolutional network

Predicted Depth Image:
1 x H x W

Predicted Depth Image:
1 x H x W

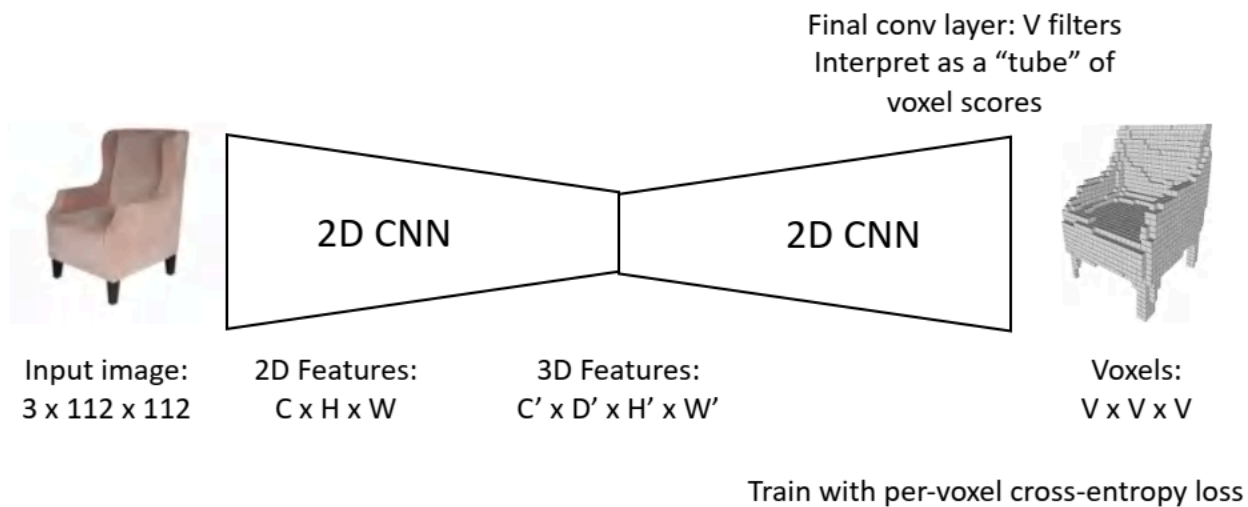
Per-Pixel Loss (Scale invariant)

Eigen, Puhrsch, and Fergus, "Depth Map Prediction from a Single Image using a Multi-Scale Deep Network", NeurIPS 2014
Eigen and Fergus, "Predicting Depth, Surface Normals and Semantic Labels with a Common Multi-Scale Convolutional Architecture", ICCV 2015

Voxel Grid

- 用一个V x V x V的占用网格来表示一个形状
- 就像Mask R-CNN中的分割掩码一样，但在三维空间中！
- （优点）概念简单：只是一个三维网格！
- （缺点）需要高空间分辨率来捕捉精细结构
- （缺点）扩展到高分辨率并非易事！

Generating Voxel Shapes: "Voxel Tubes"



Justin Johnson

Lecture 23 - 21

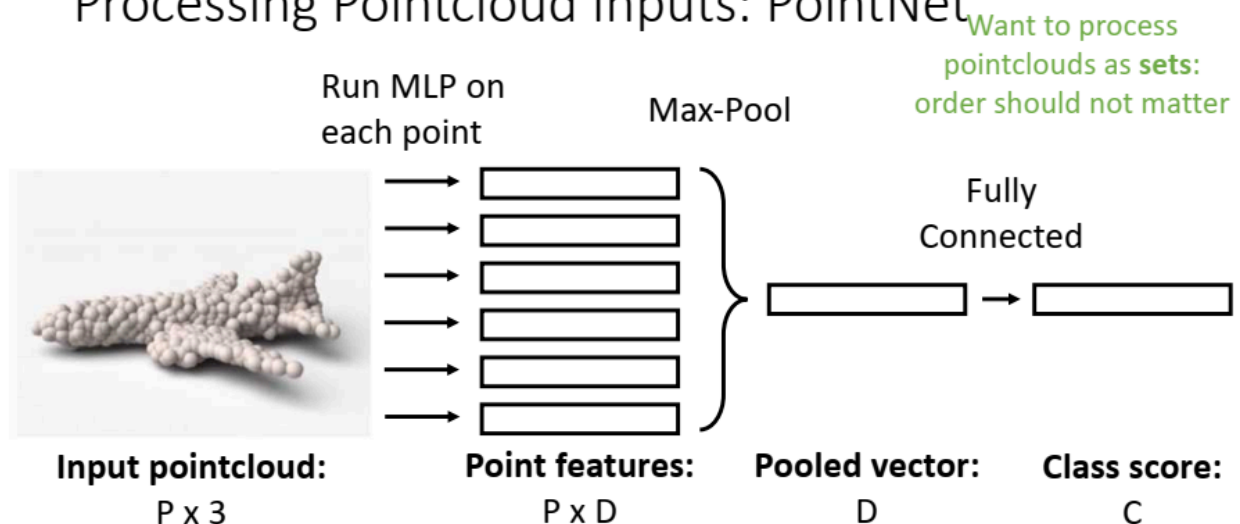
April 11, 2022

需要大量内存

Pointcloud

- 将形状表示为三维空间中的一组 P 个点
- (+) 可以在不使用大量点的情况下表示精细结构
- (-) 需要新的架构、损失函数等
- (-) 没有明确表示形状的表面：提取用于渲染或其他应用的网格需要进行后处理

Processing Pointcloud Inputs: PointNet



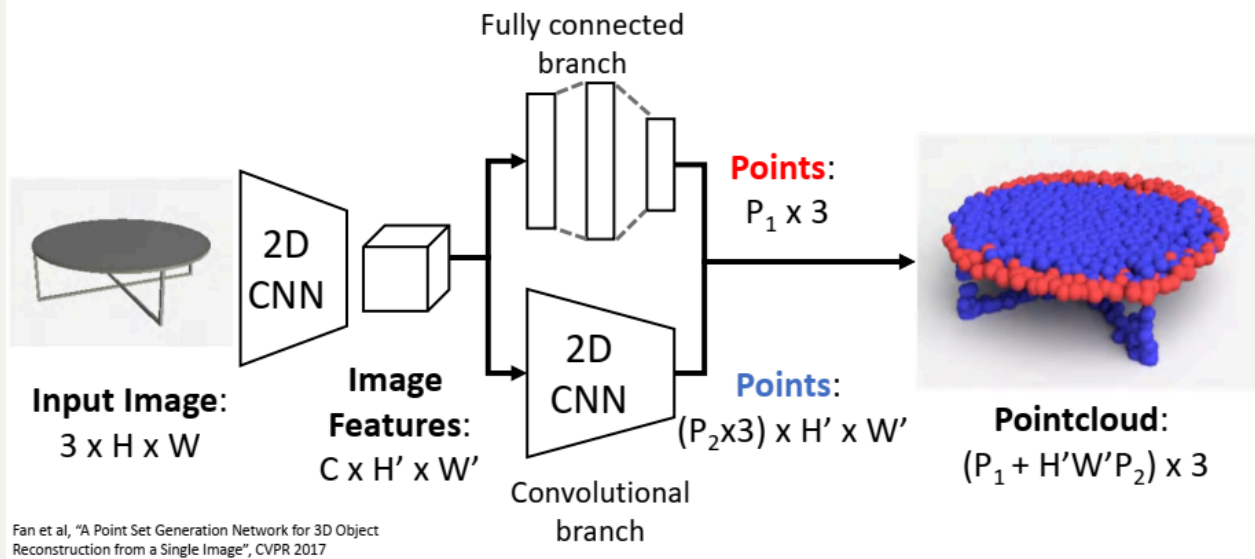
Qi et al, "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation", CVPR 2017
Qi et al, "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space", NeurIPS 2017

Justin Johnson

Lecture 23 - 26

April 11, 2022

Generating Pointcloud Outputs



Justin Johnson

Lecture 23 - 27

April 11, 2022

Loss F: Chamfer distance (两个点集的最近邻距离和, 衡量形状差异)

$$d_{CD}(A, B) = \sum_{a \in A} \min_{b \in B} \|a - b\|^2 + \sum_{b \in B} \min_{a \in A} \|a - b\|^2$$

Mesh

将 3D 形状表示为一组三角形

顶点: 3D 空间中 V 个点的集合

面: 顶点上的三角形集合

(+) 图形的标准表示

(+) 明确表示 3D 形状

(+) Adaptive: 可以非常高效地表示平面, 也可以为细节丰富的区域分配更多的面

(+) 可以在顶点上附加数据, 并在整个表面上进行插值: RGB 颜色、纹理坐标、法向量等

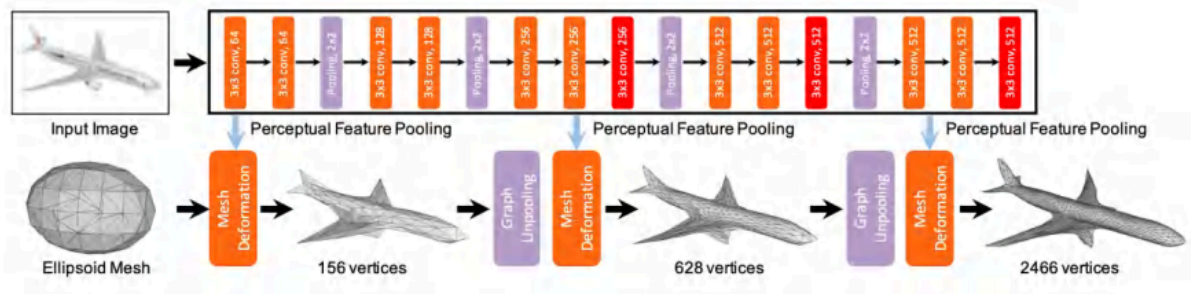
Predicting Meshes: Pixel2Mesh

Input: Single RGB Image of an object

Key ideas:

Iterative Refinement
Graph Convolution
Vertex Aligned-Features
Chamfer Loss Function

Output: Triangle mesh for the object



Wang et al, "Pixel2Mesh: Generating 3D Mesh Models from Single RGB Images", ECCV 2018

Justin Johnson

Lecture 23 - 38

April 11, 2022

1.迭代优化Idea #1: Iterative mesh refinement

Start from initial ellipsoid mesh Network predicts offsets for each vertex Repeat.

2.图卷积

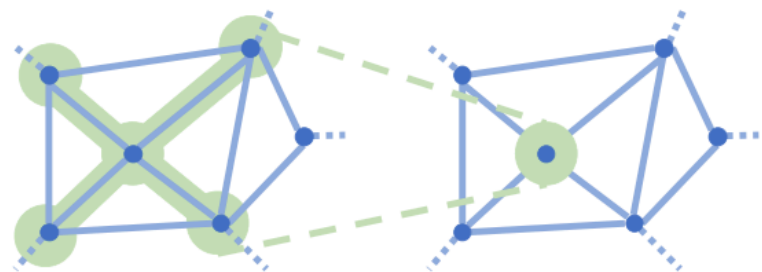
Predicting Triangle Meshes: Graph Convolution

$$f'_i = W_0 f_i + \sum_{j \in N(i)} W_1 f_j$$

Vertex v_i has feature f_i

New feature f'_i for vertex v_i depends on feature of neighboring vertices $N(i)$

Use same weights W_0 and W_1 to compute all outputs



Justin Johnson

Lecture 23 - 40

April 11, 2022

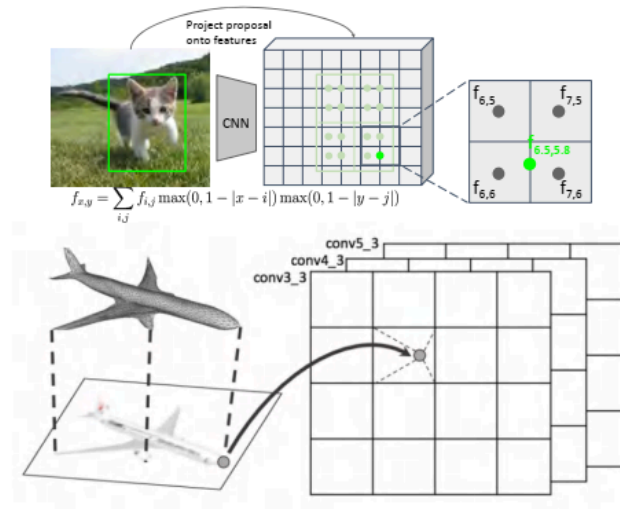
Predicting Triangle Meshes: Vertex-Aligned Features

Idea #2: Aligned vertex features

For each vertex of the mesh:

- Use camera information to project onto image plane
- Use bilinear interpolation to sample a CNN feature

Similar to RoI-Align operation from detection: maintains alignment between input image and feature vectors



Wang et al, "Pixel2Mesh: Generating 3D Mesh Models from Single RGB Images", ECCV 2018

Justin Johnson

Lecture 23 - 44

April 11, 2022

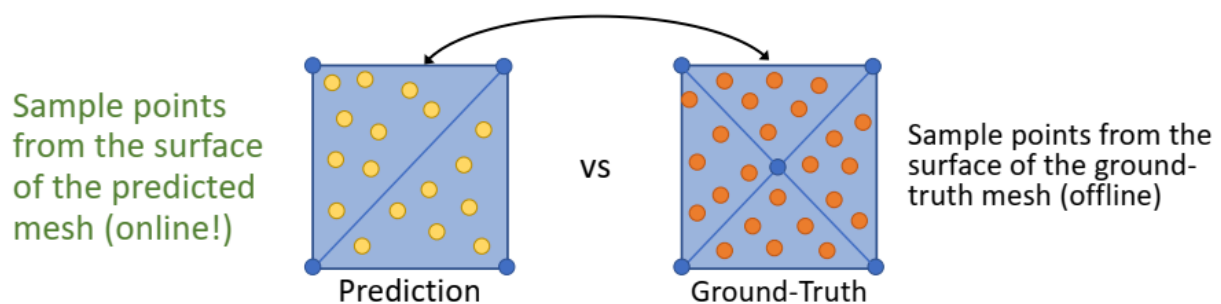
Loss F : 转换成点云再计算

Predicting Meshes: Loss Function

Problem: Need to sample online! Must be efficient!

Problem: Need to backprop through sampling!

Loss = Chamfer distance between **predicted samples** and **ground-truth samples**



Smith et al, "GEOMETRICS: Exploiting Geometric Structure for Graph-Encoded Objects", ICML 2019

Justin Johnson

Lecture 23 - 51

April 11, 2022

Implicit Functions

Learn a function to classify arbitrary 3D points as inside / outside the shape

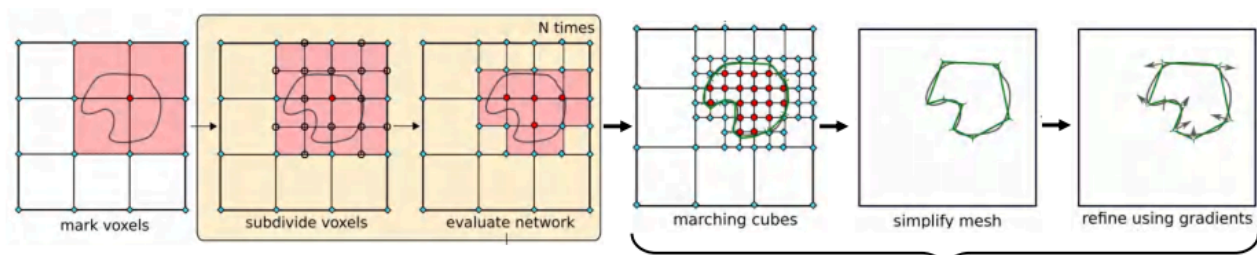
same: signed distance function (SDF)

3D Shape Representations: Implicit Functions

Learn a function to classify arbitrary 3D points as inside / outside the shape

$$o : \mathbb{R}^3 \rightarrow \{0, 1\}$$

The surface of the 3D object is the level set $\{x : o(x) = \frac{1}{2}\}$



Mescheder et al, "Occupancy Networks: Learning 3D Reconstruction in Function Space", CVPR 2019

Extracting explicit shape outputs requires post-processing

Justin Johnson

Lecture 23 - 70

April 11, 2022

Neural Radiance Fields (NeRF) for View Synthesis

神经网络表示一个函数 $f(x, d) \rightarrow (c, \sigma)$

输入:

- 空间中的某个点 $x = (x, y, z)$
- 观察方向 d

输出:

- 颜色 $c = (r, g, b)$
- 密度 σ (相当于物体是否存在)

体绘制

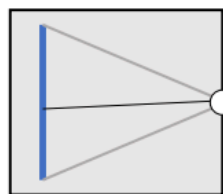
Volume Rendering

Abstract away light sources, objects.

For each point in space, need to know:

(1) How much light does it emit?

(2) How opaque is it? $\sigma \in [0,1]$



Ray origin

Parameterize each ray as origin plus

direction: $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$

Volume Density is $\sigma(\mathbf{p}) \in [0,1]$

Color that a point $\mathbf{p}$ emits in direction $\mathbf{d}$ is $c(\mathbf{p}, \mathbf{d}) \in [0,1]^3$

Color observed by the camera given by **volume rendering equation**:

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) c(\mathbf{r}(t), \mathbf{d}) dt$$

$$T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right)$$

Approximate integrals with a set of samples:

$$C(\mathbf{r}) \approx \sum_{i=1}^N T_i (1 - \exp(-\sigma_i \delta_i)) c_i$$

$$T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right)$$

Mildenhall et al, "Representing Scenes as Neural Radiance Fields for View Synthesis", ECCV 2020

Justin Johnson

Lecture 23 - 93

April 11, 2022

- σ_i : 第 i 个采样点的**体密度**, c_i 为该点的颜色。
- T_i : 光线到达第 i 个采样点的**透射率**, 公式为: $T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right)$ (表示前 $i-1$ 个采样段对光线的总吸收后, 剩余的透射比例)。
- $(1 - \exp(-\sigma_i \delta_i))$: 第 i 个采样段对光线的贡献比例 (近似该段的不透明度累积效果, 当 $\sigma_i \delta_i$ 较小时约等于 $\sigma_i \delta_i$)

Fully-connected network: Input position $\mathbf{x}$ and direction $\mathbf{d}$, and output volume density and RGB color

use positional encodings传入网络

速度慢!

Broadcast

广播允许在不同形状的张量之间进行逐元素操作, 无需显式复制数据。规则如下:

1. **维度对齐**: 从后往前比较张量的维度。

2. **维度兼容**: 每个维度的大小相等, 或其中一个为1, 或其中一个张量在该维度不存在。
3. **扩展操作**: 较小张量在缺失维度上扩展, 匹配较大张量的形状。

示例:

```
1 A = torch.rand(3, 1) # shape (3, 1)
2 B = torch.rand(1, 4) # shape (1, 4)
3 C = A + B             # 广播后为 (3, 4)
```

**** 2. 识别可向量化的操作 ****

常见可向量化的场景:

- **逐样本、逐类别的循环**: 如计算得分、边距、梯度。
- **条件判断**: 如 `if margin > 0`, 可用布尔掩码替代。
- **索引操作**: 如提取正确类别的得分。

示例: 结构化SVM损失函数

1. 计算得分矩阵:

```
1 scores = X @ W # (N, C)
```

2. 提取正确得分:

```
1 correct_scores = scores[range(N), y].view(-1, 1) # (N, 1)
```

3. 计算边距:

```
1 margin = scores - correct_scores + 1 # 广播到 (N, C)
2 margin = torch.clamp(margin, min=0) # 替代 max(0, margin)
```

4. 掩码处理正确类别:

```
1 margin[range(N), y] = 0 # 正确类别的边距置零
```

3. 构造掩码矩阵

通过布尔索引或浮点掩码替代条件判断。

梯度计算示例：

1. 生成边距掩码：

```
1 mask = (margin > 0).float() # (N, C)，边距>0的位置为1，否则为0
```

2. 处理正确类别的梯度：

```
1 correct_mask = -mask.sum(dim=1) # 每个样本的梯度累加和取反
2 mask[range(N), y] = correct_mask # 正确类别的梯度更新
```

3. 计算梯度矩阵：

```
1 dw = (X.T @ mask) / N + 2 * reg * W # 矩阵乘法替代逐样本累加
```

** 常见错误**

错误类型	原因	解决方法
维度不匹配	张量形状不符合广播规则	使用 <code>view()</code> 或 <code>unsqueeze()</code> 调整维度
索引越界	标签 <code>y</code> 超出类别范围	确保 <code>y</code> 中的值在 <code>[0, c-1]</code> 内
梯度方向错误	掩码符号错误	检查正确类别的梯度是否为负累加值
数值不稳定	未处理极大或极小值	使用 <code>torch.clamp()</code> 限制数值范围

- 广播核心：通过调整张量形状，利用逐元素操作替代循环。
- 关键步骤：构造掩码矩阵、维度对齐、矩阵乘法替代循环。
- 验证方法：小规模测试、数值梯度检验、单元测试。

`*/ @`

- *** 运算符**：执行的是**元素级乘法 (Element-wise Multiplication)**，也称为**哈达玛积 (Hadamard product)**。它要求对应位置的元素相乘。
- **@ 运算符**：执行的是**矩阵乘法 (Matrix Multiplication)**。它遵循线性代数中的矩阵乘法规则，涉及到行与列的点积求和。

元素级乘法应用场景：

- **缩放 (Scaling)**：将一个张量的所有元素乘以一个标量或另一个张量的对应元素。
- **应用掩码 (Applying Masks)**：将一个张量与一个布尔掩码（或 0/1 张量）进行元素级乘法，以选择性地保留或清零某些元素。
- **激活函数**：某些激活函数（如 Sigmoid、ReLU 等）在内部对每个元素独立操作，可以看作是元素级运算。
- **注意力机制中的加权值**：在注意力机制中，注意力权重（一个概率分布张量）与值（Value）张量进行元素级乘法，以实现不同值的加权。例如：`attn = attn_weights.unsqueeze(2) * A_flat`。

2. @ 运算符：矩阵乘法 (Matrix Multiplication)

形状要求：

参与运算的两个张量必须满足矩阵乘法的维度匹配规则：

- 如果 `A` 的形状是 `(..., M, K)`，`B` 的形状是 `(..., K, P)`。
- 那么 `A @ B` 的结果形状将是 `(..., M, P)`。

- **核心规则：** 第一个张量的**倒数第一个维度 (K)** 必须与第二个张量的**倒数第二个维度 (K)** 严格匹配。
- `...` 表示可以有任意数量的**批次维度 (Batch Dimensions)**。这些批次维度也必须兼容（通过广播）。
- **全连接层 (Fully Connected Layer / Linear Layer)：** 神经网络中最基本的线性变换，`输出 = 输入 @ 权重.T + 偏置`。
- **注意力机制中的查询-键点积：** 计算查询 (Query) 和键 (Key) 之间的相似度分数，这是注意力机制的核心。例如：`scores = Q @ K.transpose(-2, -1)`。
- **卷积层：** 虽然表面上是卷积操作，但在底层实现和理论上，卷积可以被视为一种特殊的矩阵乘法（例如，通过 `im2col` 操作）。
- **循环神经网络 (RNN/LSTM/GRU) 内部：** 许多门控机制和状态更新都涉及矩阵乘法。

3. 关键区别总结

特征	* (元素级乘法)	@ (矩阵乘法)
数学运算	对应位置元素相乘 (Hadamard Product)	线性代数中的矩阵乘法 (点积求和)
核心目的	逐元素缩放、筛选、组合	线性变换、特征映射、相似度计算、信息聚合
形状要求	兼容广播规则 (维度大小相同或为 1)	内维必须匹配 (第一个张量倒数第一维 = 第二个张量倒数第二维)；批次维兼容广播
结果形状	广播后的最大形状	由矩阵乘法规则决定 (例如: $(M, K) @ (K, P) \rightarrow (M, P)$)
PyTorch 函数	<code>torch.mul()</code>	<code>torch.matmul()</code> , <code>torch.bmm()</code>

1 `torch.masked_fill(mask, value)`

- 参数
 - `mask`: 与原张量形状相同的布尔张量 (`bool` 类型), `True` 表示对应位置需要被填充。
 - `value`: 要填充的值 (可以是标量或与原张量形状兼容的张量)。
- 返回值: 一个新的张量, 其中掩码为 `True` 的位置被 `value` 替换, 其他位置保持原张量的值

Regular Expression

```
import re
```

函数	功能描述
<code>re.match(pattern, string)</code>	从字符串 开头 匹配模式, 成功返回 <code>Match</code> 对象, 否则返回 <code>None</code> 。
<code>re.search(pattern, string)</code>	在整个字符串中查找 第一个 匹配项, 成功返回 <code>Match</code> 对象, 否则返回 <code>None</code> 。
<code>re.findall(pattern, string)</code>	查找所有匹配的子串, 返回 列表 (包含所有匹配结果)。
<code>re.sub(pattern, repl, string)</code>	替换所有匹配的子串, <code>repl</code> 为替换后的内容, 返回新字符串。
<code>re.split(pattern, string)</code>	根据匹配的子串分割字符串, 返回 列表 。

正则表达式基本语法

些常用的特殊字符 (元字符) 和语法:

- `.` (点号): 匹配除换行符 `\n` 之外的任何单个字符。

- **^ (脱字符)**: 匹配字符串的开头。
- **\$ (美元符号)**: 匹配字符串的结尾。
- *** (星号)**: 匹配前一个字符零次或多次。
- **+** (加号): 匹配前一个字符一次或多次。
- **?** (问号): 匹配前一个字符零次或一次 (使其成为可选的), 或使量词变为非贪婪模式。
- **{n}**: 匹配前一个字符恰好 **n** 次。
- **{n,m}**: 匹配前一个字符至少 **n** 次, 至多 **m** 次。
- **[] (方括号)**: 定义一个字符集。匹配方括号中任何一个字符 (例如 **[abc]** 匹配 'a', 'b', 或 'c')。
 - **[a-z]**: 匹配任何小写字母。
 - **[0-9]**: 匹配任何数字。
 - **[^abc]**: 匹配除 'a', 'b', 'c' 之外的任何字符。
- **| (竖线)**: 逻辑或, 匹配 **|** 符号前或后的模式 (例如 **cat|dog** 匹配 "cat" 或 "dog")。
- **() (圆括号)**: 用于分组模式, 可以捕获匹配的子串。
- **\ (反斜杠)**: 用于转义特殊字符, 或引入特殊序列。
 - **\d**: 匹配任何数字 (等同于 **[0-9]**)。
 - **\w**: 匹配任何字母、数字或下划线 (等同于 **[a-zA-Z0-9_]**)。
 - **\s**: 匹配任何空白字符 (空格、制表符、换行符等)。
 - **\b**: 匹配单词边界。
 - **\.**: 匹配实际的点号字符。
 - *****: 匹配实际的星号字符。
- **匹配数字**: **r'\d'** 匹配单个数字, **r'\d+'** 匹配一个或多个数字。

```
1 import re
2 print(re.findall(r'\d+', '年龄: 25, 身高: 180')) # 输出: ['25', '180']
```

- 匹配字母: `r'[a-zA-Z]'` 匹配单个字母, `r'[a-zA-Z]+'` 匹配单词。python

```
1 print(re.findall(r'[A-Za-z]+', 'Hello 世界, Python')) # 输出: ['Hello', 'Python']
```

- 替换操作: 将数字替换为 `*`。

```
1 print(re.sub(r'\d+', '*', '密码: 123456')) # 输出: '密码: *'
```

- 分割操作: 用逗号或空格分割字符串。

```
1 print(re.split(r'[ ,]', 'a,b c,d')) # 输出: ['a', 'b', 'c', 'd']
```

PyTorch 的 `torch.nn` 模块 (通常简称为 `nn`) 提供了构建神经网络的核心组件, 包含大量用于定义层、损失函数、激活函数等的类和方法。以下是 `nn` 中最常用的几类方法/组件的详细讲解:

常用函数

`torch.nn`

一、网络层 (Layers)

网络层是构建神经网络的基础模块, 用于对输入进行特征转换。

1. 线性层（全连接层）： `nn.Linear`

- **功能：**实现线性变换 $y = xW^T + b$
- **参数：**`in_features`（输入特征数）、`out_features`（输出特征数）、`bias`（是否使用偏置，默认 `True`）
- **示例：**

```
1 linear = nn.Linear(in_features=100, out_features=10) #  
    100→10维  
2 x = torch.randn(32, 100) # 32个样本，每个100维  
3 output = linear(x) # 输出形状: (32, 10)
```

2. 卷积层： `nn.Conv2d`（2D卷积，图像常用）

- **功能：**通过滑动卷积核提取局部特征
- **参数：**`in_channels`（输入通道数）、`out_channels`（输出通道数）、`kernel_size`（卷积核大小）、`stride`（步长，默认1）、`padding`（填充，默认0）
- **示例：**

```
1 conv = nn.Conv2d(in_channels=3, out_channels=16,  
    kernel_size=3, stride=1, padding=1)  
2 x = torch.randn(32, 3, 224, 224) # 32张3通道224×224图像  
3 output = conv(x) # 输出形状: (32, 16, 224, 224) (因  
    padding=1保持尺寸)
```

3. 循环层： `nn.LSTM` / `nn.GRU`（处理序列数据）

- **功能：**处理时序数据，捕捉序列依赖关系
- **参数：**`input_size`（输入特征数）、`hidden_size`（隐藏层维度）、`num_layers`（层数，默认1）、`batch_first`（是否批量维度在前，默认 `False`）
- **示例：**

```

1 lstm = nn.LSTM(input_size=50, hidden_size=100,
  batch_first=True)
2 x = torch.randn(32, 10, 50) # 32个样本, 每个10个时间步, 每步
  50维
3 output, (h_n, c_n) = lstm(x) # output形状: (32, 10, 100)

```

4. 嵌入层: `nn.Embedding` (用于离散特征映射)

- 功能: 将离散索引 (如词ID) 映射为低维向量
- 参数: `num_embeddings` (离散值总数)、`embedding_dim` (嵌入维度)
- 示例:

```

1 embed = nn.Embedding(num_embeddings=10000,
  embedding_dim=128) # 10000个词, 128维嵌入
2 x = torch.randint(0, 10000, (32, 20)) # 32个样本, 每个20个
  词ID
3 output = embed(x) # 输出形状: (32, 20, 128)

```

二、激活函数 (Activation Functions)

激活函数为网络引入非线性, 使模型能拟合复杂关系。

1. `nn.ReLU`: 修正线性单元

- 功能: $y = \max(0, x)$, 缓解梯度消失, 应用最广泛
- 示例:

```

1 relu = nn.ReLU()
2 x = torch.randn(32, 100)
3 output = relu(x) # 所有负数变为0

```

2. `nn.Sigmoid`: S形函数

- 功能: $y = 1/(1 + e^{-x})$, 输出范围 (0, 1), 常用于二分类输出层
- 示例:

```
1 sigmoid = nn.Sigmoid()
2 output = sigmoid(torch.tensor([-1.0, 0.0, 1.0])) # 输出:
           [0.2689, 0.5, 0.7311]
```

3. nn.Softmax: 归一化函数

- 功能: 将向量转换为概率分布 (和为1), 常用于多分类输出层
- 参数: `dim` (指定归一化的维度)
- 示例:

```
1 softmax = nn.Softmax(dim=1)
2 x = torch.randn(32, 10) # 32个样本, 10个类别得分
3 output = softmax(x) # 每行和为1, 形状不变
```

三、损失函数 (Loss Functions)

损失函数用于衡量模型预测与真实标签的差异, 是训练的“目标”。

1. nn.CrossEntropyLoss: 交叉熵损失

- 功能: 结合 `nn.LogSoftmax` 和 `nn.NLLLoss`, 直接用于多分类任务 (标签为整数)
- 示例:

```
1 criterion = nn.CrossEntropyLoss()
2 pred = torch.randn(32, 10) # 32个样本, 10类预测得分
3 label = torch.randint(0, 10, (32,)) # 真实标签 (0~9)
4 loss = criterion(pred, label) # 计算损失
```

2. nn.MSELoss: 均方误差损失

- 功能: $\text{loss} = \frac{1}{N} \sum (y_{\text{pred}} - y_{\text{true}})^2$, 用于回归任务
- 示例:

```
1 criterion = nn.MSELoss()
2 pred = torch.randn(32, 1) # 回归预测值
3 label = torch.randn(32, 1) # 真实值
4 loss = criterion(pred, label)
```

3. nn.BCELoss：二分类交叉熵损失

- 功能：用于二分类（标签为0/1，且预测值需经 `Sigmoid` 处理为概率）
- 示例：

```
1 criterion = nn.BCELoss()
2 pred = torch.sigmoid(torch.randn(32, 1)) # 经Sigmoid的预测
   概率
3 label = torch.randint(0, 2, (32, 1)).float() # 真实标签（0
   或1，需为float）
4 loss = criterion(pred, label)
```

四、容器（Containers）

容器用于组织多个模块，管理网络结构。

1. nn.Sequential：顺序容器

- 功能：按顺序包装多个模块，前向传播时依次执行（简化代码）
- 示例：

```
1 model = nn.Sequential(
2     nn.Linear(100, 64),
3     nn.ReLU(),
4     nn.Linear(64, 10),
5     nn.Softmax(dim=1)
6 )
```

2. `nn.ModuleList`：模块列表

- **功能**：存储多个模块，支持动态添加/访问，需手动定义前向传播（见前文详细讲解）

五、正则化（Regularization）

防止过拟合的技术。

1. `nn.Dropout`：随机失活

- **功能**：训练时随机将部分神经元输出设为0，降低过拟合风险
- **参数**：`p`（失活概率，如 `p=0.5` 表示50%概率失活）
- **示例**：

```
1 dropout = nn.Dropout(p=0.5)
2 x = torch.randn(32, 100)
3 output = dropout(x) # 训练时50%元素被置0，测试时无变化
```

2. `nn.BatchNorm2d`：批归一化

- **功能**：对批次内数据进行归一化（均值0，方差1），加速训练收敛，增强稳定性
- **参数**：`num_features`（特征数，与输入通道数一致）
- **示例**：

```
1 bn = nn.BatchNorm2d(num_features=16) # 对应16通道的卷积输出
2 x = torch.randn(32, 16, 224, 224)
3 output = bn(x) # 对每个通道进行批归一化
```

六、其他常用方法

- `nn.Flatten`：将多维张量展平为一维（如CNN接全连接层前使用）
- `nn.MaxPool2d` / `nn.AvgPool2d`：池化层，用于降维或提取关键特征
- `nn.Transformer`：完整的Transformer模型，包含自注意力机制

`torch.nn` 模块提供了构建神经网络所需的几乎所有基础组件，从简单的线性层、激活函数，到复杂的循环层、注意力机制，再到损失函数和正则化工具。掌握这些组件的功能和用法，是搭建和训练深度学习模型的基础。实际使用中，通常会通过继承 `nn.Module` 类，组合这些组件定义自定义网络结构。

torch.triu

```
1 torch.triu(input, diagonal=0)
```

参数：

- `input`：输入张量（通常是 2D 矩阵，也支持高维张量）。

- `diagonal`

：指定对角线偏移量，控制保留哪条对角线及以上的元素（默认值为 0）：

- `diagonal=0`：保留主对角线及以上的元素（默认行为）。
- `diagonal>0`：保留主对角线上方元素（如 `diagonal=1` 保留主对角线 + 1 及以上）。
- `diagonal<0`：保留主对角线下方元素（此时类似下三角矩阵，但函数名仍为 `triu`）。

返回值：与输入形状相同的张量，上三角部分保留原值，其余部分为 0

torch.linspace

```
1 torch.linspace(start, end, steps=100, out=None, dtype=None,
  device=None, requires_grad=False)
```

参数说明:

- `start`: 序列的起始值 (标量)。
- `end`: 序列的结束值 (标量)。
- `steps`: 生成的点数数量 (默认值为 100, 必须为非负整数)。
- 其他参数 (`out`、`dtype` 等): 用于指定输出张量、数据类型等, 一般无需手动设置。

返回值: 形状为 `(steps,)` 的一维张量, 包含从 `start` 到 `end` 均匀分布的 `steps` 个点

`torch.arange`

```
1 torch.arange(start=0, end, step=1, out=None, dtype=None,
  device=None, requires_grad=False)
```

核心参数:

- `start`: 序列的起始值 (可选, 默认值为 0)。
- `end`: 序列的结束值 (必填), 生成的序列**不包含** `end`。
- `step`: 相邻元素的步长 (可选, 默认值为 1), 可以是整数或浮点数。
- `dtype`: 输出张量的数据类型 (如 `torch.int32`、`torch.float32`, 默认自动推断)。

特性	TORCH.ARANGE	TORCH.Linspace
参数逻辑	基于「起始值、结束值、步长」生成序列	基于「起始值、结束值、点数」生成序列

特性	TORCH.ARANGE	TORCH.Linspace
是否包含 终点	通常不包含（除非刚好能整除）	始终包含终点
序列长度	由 <code>(end - start) / step</code> 自动计算（不固定）	由 <code>steps</code> 参数固定指定
典型用途	生成整数索引、固定步长的序列	生成均匀分布的采样点（如坐标、参数网格）

torch.where

```
1 torch.where(condition, x, y)
```

- `condition`: 布尔张量 (`True/False`)，用于指定选择规则。
- `x`: 当 `condition` 为 `True` 时，选择该张量的对应元素。
- `y`: 当 `condition` 为 `False` 时，选择该张量的对应元素。

要求: `condition`、`x`、`y` 必须具有相同的形状，或可广播为相同形状

torch.any

用于判断张量中是否存在至少一个满足条件的元素的方法。它返回一个布尔值（或布尔张量）

```
1 torch.Tensor.any(dim=None, keepdim=False)
```

- `dim`: 可选参数，指定要判断的维度。若不指定（默认 `None`），则对整个张量进行判断，返回一个单个布尔值。

- `keepdim`: 若为 `True`, 则保留判断后的维度 (形状中该维度大小为 1); 若为 `False` (默认), 则删除该维度

```
1 # 二维张量: shape (2, 3)
2 tensor = torch.tensor([[False, True, False],
3                         [False, False, False]])
4
5 # 按 dim=0 (行方向) 判断: 检查每一列是否有 True
6 print(tensor.any(dim=0)) # 输出: tensor([False,  True,  False])
7
8 # 按 dim=1 (列方向) 判断: 检查每一行是否有 True
9 print(tensor.any(dim=1)) # 输出: tensor([ True,  False])
```

`torch.clamp`

限制张量元素值范围

```
1 torch.clamp(input, min=None, max=None, *, out=None)
```

- `input`: 输入张量。
- `min`: 下界 (可选), 所有小于 `min` 的元素会被设为 `min`。
- `max`: 上界 (可选), 所有大于 `max` 的元素会被设为 `max`。
- `out`: 输出张量 (可选), 用于指定结果的存储位置

`torch.randperm(n)`

生成 0 到 n-1 的随机排列

```
1 torch.topk(input, k, dim=None, largest=True, sorted=True, *,
2 out=None)
```

- `input`: 输入张量 (可以是多维)。

- `k`: 要选取的前 `k` 个元素（必须是正整数，且 $\leq$ 输入维度的大小）。
- `dim`: 指定在哪个维度上选取（默认是最后一个维度）。
- `largest`: 若为 `True`（默认），选取最大的 `k` 个元素；若为 `False`，选取最小的 `k` 个元素。
- `sorted`: 若为 `True`（默认），返回的元素按从大到小排序；若为 `False`，顺序不确定
- `return` 元素及其索引

torch.repeat()

```
1 Tensor.repeat(*sizes)
```

`*sizes`: 一个或多个整数，指定每个维度的重复次数。

- 若输入张量是 `n` 维，则 `sizes` 必须包含 `n` 个整数（或通过补 1 适配维度）。
- 例如，对 2D 张量 `x`，`x.repeat(a, b)` 表示第 0 维重复 `a` 次，第 1 维重复 `b` 次

torch.cat

```
1 torch.cat(tensors, dim=0, out=None)
```

- `tensors`: 需要拼接的张量序列（如列表或元组），要求除拼接维度外，其他维度的形状必须相同。
- `dim`: 指定拼接的维度（从 0 开始计数）。
- `out`: 可选参数，用于指定输出张量

与 `stack` 的区别: `torch.stack()` 会新增一个维度并拼接，而 `torch.cat()` 是在已有维度上合并（不新增维度）

```

1 torchvision.ops.roi_align(
2     input,          # 特征图, 形状为 [N, C, H, W]
3     boxes,          # ROI 区域, 形状为 [M, 5], 格式为 [batch_idx, x1,
                        y1, x2, y2]
4     output_size,    # 输出特征图大小, 如 (7,7)
5     spatial_scale=1.0, # 特征图与原始图像的缩放比例 (特征图尺寸/原始图像
                        尺寸)
6     sampling_ratio=-1 # 每个子区域的采样点数, -1 表示自动计算
7 )

```

torch.rand

生成服从区间 $[0, 1)$ 上均匀分布的随机张量

```
torch.rand(*size, out=None, dtype=None, layout=torch.strided, device=None,
requires_grad=False)
```

函数	分布类型	取值范围	典型用途
<code>torch.rand</code>	均匀分布	$[0, 1)$	参数初始化、随机掩码
<code>torch.randn</code>	标准正态分布	$(-\infty, +\infty)$ (均值 0, 方差 1)	模拟噪声、特征初始化
<code>torch.randint(low, high, size)</code>	整数均匀分布	$[low, high)$	生成随机索引、标签
<code>torch.randperm(n)</code>	整数排列	$[0, n-1]$ 的随机排列	

工具库

- **PyTorch Lightning**: 简化训练循环。

```

1 trainer = pl.Trainer(max_epochs=10, gpus=1)
2 trainer.fit(model, datamodule)

```

- **ONNX 导出**: 模型跨平台部署。

```
1 torch.onnx.export(model, dummy_input, "model.onnx")
```

模型轻量化

- **知识蒸馏**: 用大模型（教师）训练小模型（学生）。
- **量化**: 降低权重精度 (FP32 $\rightarrow$ INT8)。

```
1 model_quantized =  
    torch.quantization.quantize_dynamic(model, {nn.Linear},  
    dtype=torch.qint8)
```